

# Banco de Dados

Prof<sup>a</sup> Angelica Nakagawa Lima  
angelica.lima@ibta.edu.br

# SELECT

## Consultando dados - SELECT

- O comando SELECT permite a seleção e manipulação, para visualização das informações armazenadas no banco de dados.
- Veja a sintaxe básica:

```
SELECT col1, col2, coln  
FROM nome da tabela  
WHERE condição
```

# Consultando datos - Ejemplos

- Podemos seleccionar ...

```
SELECT * FROM departments;
```

|    | DEPARTMENT_ID | DEPARTMENT_NAME      | MANAGER_ID | LOCATION_ID |
|----|---------------|----------------------|------------|-------------|
| 1  | 10            | Administration       | 200        | 1700        |
| 2  | 20            | Marketing            | 201        | 1800        |
| 3  | 30            | Purchasing           | 114        | 1700        |
| 4  | 40            | Human Resources      | 203        | 2400        |
| 5  | 50            | Shipping             | 121        | 1500        |
| 6  | 60            | IT                   | 103        | 1400        |
| 7  | 70            | Public Relations     | 204        | 2700        |
| 8  | 80            | Sales                | 145        | 2500        |
| 9  | 90            | Executive            | 100        | 1700        |
| 10 | 100           | Finance              | 108        | 1700        |
| 11 | 110           | Accounting           | 205        | 1700        |
| 12 | 120           | Treasury             | (null)     | 1700        |
| 13 | 130           | Corporate Tax        | (null)     | 1700        |
| 14 | 140           | Control And Credit   | (null)     | 1700        |
| 15 | 150           | Shareholder Services | (null)     | 1700        |

# Consultando datos - Ejemplos

- Podemos seleccionar ...

**SELECT** first\_name, email, hire\_date **FROM**  
employees;

|    | FIRST_NAME  | EMAIL    | HIRE_DATE |
|----|-------------|----------|-----------|
| 1  | Steven      | SKING    | 17/06/87  |
| 2  | Neena       | NKOCHHAR | 21/09/89  |
| 3  | Lex         | LDEHAAN  | 13/01/93  |
| 4  | Alexander   | AHUNOLD  | 03/01/90  |
| 5  | Bruce       | BERNST   | 21/05/91  |
| 6  | David       | DAUSTIN  | 25/06/97  |
| 7  | Valli       | VPATABAL | 05/02/98  |
| 8  | Diana       | DLORENTZ | 07/02/99  |
| 9  | Nancy       | NGREENBE | 17/08/94  |
| 10 | Daniel      | DFAVIET  | 16/08/94  |
| 11 | John        | JCHEN    | 28/09/97  |
| 12 | Ismael      | ISCIARRA | 30/09/97  |
| 13 | Jose Manuel | JMURMAN  | 07/03/98  |
| 14 | Luis        | LPOPP    | 07/12/99  |
| 15 | Den         | DRAPHEAL | 07/12/94  |
| 16 | Alexander   | AKHOO    | 18/05/95  |
| 17 | Shelli      | SBAIDA   | 24/12/97  |

## Consultando datos - Ejemplos

- Podemos seleccionar ...

```
SELECT first_name, email , salary, salary*12  
FROM employees;
```

|    | FIRST_NAME  | EMAIL    | SALARY | SALARY*12 |
|----|-------------|----------|--------|-----------|
| 1  | Steven      | SKING    | 24000  | 288000    |
| 2  | Neena       | NKOCHHAR | 17000  | 204000    |
| 3  | Lex         | LDEHAAN  | 17000  | 204000    |
| 4  | Alexander   | AHUNOLD  | 9000   | 108000    |
| 5  | Bruce       | BERNST   | 6000   | 72000     |
| 6  | David       | DAUSTIN  | 4800   | 57600     |
| 7  | Valli       | VPATABAL | 4800   | 57600     |
| 8  | Diana       | DLORENTZ | 4200   | 50400     |
| 9  | Nancy       | NGREENBE | 12000  | 144000    |
| 10 | Daniel      | DFAVIET  | 9000   | 108000    |
| 11 | John        | JCHEN    | 8200   | 98400     |
| 12 | Ismael      | ISCIARRA | 7700   | 92400     |
| 13 | Jose Manuel | JMURMAN  | 7800   | 93600     |
| 14 | Luis        | LPOPP    | 6900   | 82800     |
| 15 | Den         | DRAPHEAL | 11000  | 132000    |
| 16 | Alexander   | AKHOO    | 3100   | 37200     |

# Consultando datos - Exemplos

- Podemos seleccionar ...

```
SELECT first_name, email, salary "Salário",  
salary*12 "Salario Anual" FROM employees;
```

Alias  
(apelido)

Alias  
(apelido)

|    | FIRST_NAME  | EMAIL    | Salário | Salario Anual |
|----|-------------|----------|---------|---------------|
| 1  | Steven      | SKING    | 24000   | 288000        |
| 2  | Neena       | NKOCHHAR | 17000   | 204000        |
| 3  | Lex         | LDEHAAN  | 17000   | 204000        |
| 4  | Alexander   | AHUNOLD  | 9000    | 108000        |
| 5  | Bruce       | BERNST   | 6000    | 72000         |
| 6  | David       | DAUSTIN  | 4800    | 57600         |
| 7  | Valli       | VPATABAL | 4800    | 57600         |
| 8  | Diana       | DLORENTZ | 4200    | 50400         |
| 9  | Nancy       | NGREENBE | 12000   | 144000        |
| 10 | Daniel      | DFAVIET  | 9000    | 108000        |
| 11 | John        | JCHEN    | 8200    | 98400         |
| 12 | Ismael      | ISCIARRA | 7700    | 92400         |
| 13 | Jose Manuel | JMURMAN  | 7800    | 93600         |
| 14 | Luis        | LPOPP    | 6900    | 82800         |
| 15 | Den         | DRAPHEAL | 11000   | 132000        |
| 16 | Alexander   | AKHOO    | 3100    | 37200         |
| 17 | Shelli      | SBAIDA   | 2900    | 34800         |



# Consultando dados - Exemplos

- Podemos selecionar ...

Concatenação

```
SELECT first_name, email, 'Salário: ' || salary "Salário",  
salary*12 "Salario Anual" FROM employees;
```

Alias  
(apelido)

| FIRST_NAME     | EMAIL    | Salário        | Salario Anual |
|----------------|----------|----------------|---------------|
| 1 Steven       | SKING    | Salário: 24000 | 288000        |
| 2 Neena        | NKOCHHAR | Salário: 17000 | 204000        |
| 3 Lex          | LDEHAAN  | Salário: 17000 | 204000        |
| 4 Alexander    | AHUNOLD  | Salário: 9000  | 108000        |
| 5 Bruce        | BERNST   | Salário: 6000  | 72000         |
| 6 David        | DAUSTIN  | Salário: 4800  | 57600         |
| 7 Valli        | VPATABAL | Salário: 4800  | 57600         |
| 8 Diana        | DLORENTZ | Salário: 4200  | 50400         |
| 9 Nancy        | NGREENBE | Salário: 12000 | 144000        |
| 10 Daniel      | DFAVIET  | Salário: 9000  | 108000        |
| 11 John        | JCHEN    | Salário: 8200  | 98400         |
| 12 Ismael      | ISCIARRA | Salário: 7700  | 92400         |
| 13 Jose Manuel | JMURMAN  | Salário: 7800  | 93600         |
| 14 Luis        | LPOPP    | Salário: 6900  | 82800         |
| 15 Den         | DRAPHEAL | Salário: 11000 | 132000        |



# Consultando dados - DISTINCT

- A exibição *default* de consultas são todas as linhas, incluindo as linhas duplicadas;
- Para eliminar valores duplicados no resultado, incluimos o **DISTINCT** qualificador no comando **SELECT**.

```
SELECT department_id  
FROM employees  
ORDER BY department_id;
```

|    | DEPARTMENT_ID |
|----|---------------|
| 1  | 10            |
| 2  | 20            |
| 3  | 20            |
| 4  | 30            |
| 5  | 30            |
| 6  | 30            |
| 7  | 30            |
| 8  | 30            |
| 9  | 30            |
| 10 | 40            |
| 11 | 50            |
| 12 | 50            |
| 13 | 50            |

```
SELECT DISTINCT department_id  
FROM employees  
ORDER BY department_id;
```

|    | DEPARTME... |
|----|-------------|
| 1  | 10          |
| 2  | 20          |
| 3  | 30          |
| 4  | 40          |
| 5  | 50          |
| 6  | 60          |
| 7  | 70          |
| 8  | 80          |
| 9  | 90          |
| 10 | 100         |
| 11 | 110         |
| 12 | (null)      |

# Consultando dados - Operadores

- Expressões Aritméticas podem compor nome de colunas, valores numéricos constantes e operadores aritméticos:

| Operadores | Descrição     |
|------------|---------------|
| +          | Adição        |
| -          | Subtração     |
| *          | Multiplicação |
| /          | Divisão       |

```
SELECT first_name, email  
, salary, salary*12  
FROM employees;
```

|    | FIRST_NAME  | EMAIL    | SALARY | SALARY*12 |
|----|-------------|----------|--------|-----------|
| 1  | Steven      | SKING    | 24000  | 288000    |
| 2  | Neena       | NKOCHHAR | 17000  | 204000    |
| 3  | Lex         | LDEHAAN  | 17000  | 204000    |
| 4  | Alexander   | AHUNOLD  | 9000   | 108000    |
| 5  | Bruce       | BERNST   | 6000   | 72000     |
| 6  | David       | DAUSTIN  | 4800   | 57600     |
| 7  | Valli       | VPATABAL | 4800   | 57600     |
| 8  | Diana       | DLORENTZ | 4200   | 50400     |
| 9  | Nancy       | NGREENBE | 12000  | 144000    |
| 10 | Daniel      | DFAVIET  | 9000   | 108000    |
| 11 | John        | JCHEN    | 8200   | 98400     |
| 12 | Ismael      | ISCIARRA | 7700   | 92400     |
| 13 | Jose Manuel | JMURMAN  | 7800   | 93600     |
| 14 | Luis        | LPOPP    | 6900   | 82800     |
| 15 | Den         | DRAPHEAL | 11000  | 132000    |
| 16 | Alexander   | AKHOO    | 3100   | 37200     |

# Consultando dados - Operadores

- A Cláusula **WHERE** indica condição para um **SELECT**, no qual pode ter os seguintes operadores:

| Operadores | Descrição      |
|------------|----------------|
| =          | Igualdade      |
| <>         | Diferente      |
| >          | Maior          |
| <          | Menor          |
| >=         | Maior ou igual |
| <=         | Menor ou igual |

```
SELECT first_name, email ,  
salary, salary*12  
FROM employees  
WHERE salary < 9000;
```

|   | FIRST_NAME | EMAIL    | SALARY | SALARY*12 |
|---|------------|----------|--------|-----------|
| 1 | James      | JLANDRY  | 2400   | 28800     |
| 2 | Steven     | SMARKLE  | 2200   | 26400     |
| 3 | TJ         | TJOLSON  | 2100   | 25200     |
| 4 | Ki         | KGEE     | 2400   | 28800     |
| 5 | Hazel      | HPHILTAN | 2200   | 26400     |

# Criação de tabela a partir de uma consulta

- Este comando cria uma tabela (estrutura e dados) baseado em um comando **SELECT**.

```
CREATE TABLE Espelho_Pais  
AS SELECT * FROM Pais;
```

A screenshot of a SQL query execution window. The query bar shows 'select \* from pais;'. Below the query bar, there are tabs for 'Saída do Script' and 'Resultado da Consulta'. The 'Resultado da Consulta' tab is active, showing a table with 4 rows and 3 columns: CD\_PAIS, NM\_PAIS, and DS\_NACIONALIDADE. The data is as follows:

| CD_PAIS | NM_PAIS  | DS_NACIONALIDADE |
|---------|----------|------------------|
| 1       | 1 Brasil | ... (null)       |
| 2       | 2 França | ... (null)       |
| 3       | 3 EUA    | ... (null)       |
| 4       | 4 Chile  | ... (null)       |

A screenshot of a SQL query execution window. The query bar shows 'select \* from Espelho\_Pais;'. Below the query bar, there are tabs for 'Saída do Script' and 'Resultado da Consulta'. The 'Resultado da Consulta' tab is active, showing a table with 4 rows and 3 columns: CD\_PAIS, NM\_PAIS, and DS\_NACIONALIDADE. The data is identical to the previous table:

| CD_PAIS | NM_PAIS  | DS_NACIONALIDADE |
|---------|----------|------------------|
| 1       | 1 Brasil | ... (null)       |
| 2       | 2 França | ... (null)       |
| 3       | 3 EUA    | ... (null)       |
| 4       | 4 Chile  | ... (null)       |

## SELECT– Operadores SQL

| Operador               | Significado                       |
|------------------------|-----------------------------------|
| BETWEEN ... AND<br>... | Entre dois valores (inclusive)    |
| IN (lista)             | Comparar uma lista de valores     |
| LIKE                   | Compara um parâmetro alfanumérico |
| IS NULL                | É um valor nulo                   |

## SELECT- BETWEEN... AND...

- Compara uma faixa de valores inclusive o menor e maior valor.

```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE Salary BETWEEN 10000 AND 20000;
```

- No exemplo acima são listados os dados dos funcionários que tem salário entre 10.000 e 20.000, incluindo estes valores. Os valores devem ser informados do menor para o maior.

|    | FIRST_NAME | LAST_NAME | SALARY |
|----|------------|-----------|--------|
| 1  | Neena      | Kochhar   | 17000  |
| 2  | Lex        | De Haan   | 17000  |
| 3  | Nancy      | Greenberg | 12000  |
| 4  | Den        | Raphaely  | 11000  |
| 5  | John       | Russell   | 14000  |
| 6  | Karen      | Partners  | 13500  |
| 7  | Alberto    | Errazuriz | 12000  |
| 8  | Gerald     | Cambrault | 11000  |
| 9  | Eleni      | Zlotkey   | 10500  |
| 10 | Peter      | Tucker    | 10000  |

# SELECT- IN

- Compara os valores especificados dentro de uma lista.

```
SELECT First_name, Last_name, Salary, Manager_id  
FROM Employees  
WHERE Manager_id IN (101, 145, 149);
```

- No exemplo acima são listados os dados dos funcionários são subordinados dos gerentes indicados pelos códigos dentro do **IN**.
- Se os valores forem alfanuméricos tem que estar entre aspas simples.

| FIRST_NAME  | LAST_NAME | SALARY | MANAGER_ID |
|-------------|-----------|--------|------------|
| Nancy       | Greenberg | 12000  | 101        |
| Jennifer    | Whalen    | 4400   | 101        |
| Susan       | Mavris    | 6500   | 101        |
| Hermann     | Baer      | 10000  | 101        |
| Shelley     | Higgins   | 12000  | 101        |
| Peter       | Tucker    | 10000  | 145        |
| David       | Bernstein | 9500   | 145        |
| Peter       | Hall      | 9000   | 145        |
| Christopher | Olsen     | 8000   | 145        |
| Nanette     | Cambrault | 7500   | 145        |



## SELECT- LIKE

- Algumas vezes você precisa procurar valores que você não conhece exatamente. O operador **LIKE** permite selecionar linhas combinando parâmetros alfanuméricos. O caracter % é utilizado como coringa nas pesquisas. Veja exemplo abaixo:

Like '%cadeia'

Usar o coringa no início da cadeia permite localizar valores que terminam com ela

Like 'cadeia%'

Usar o coringa no fim da cadeia permite localizar valores que começam com ela

Like '%cadeia%'

Usar o coringa em ambos os lados da cadeia permite localizar valores que a contenham em qualquer parte do valor do campo

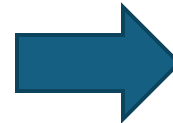
# SELECT - LIKE - Exemplos

|   | FIRST_NAME | LAST_NAME | SALARY |
|---|------------|-----------|--------|
| 1 | Shelli     | Baida     | 2900   |
| 2 | Shanta     | Vollman   | 6500   |
| 3 | Shelley    | Higgins   | 12000  |



```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE First_name LIKE 'Sh%';
```

```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE First_name LIKE '%s';
```



|   | FIRST_NAME | LAST_NAME | SALARY |
|---|------------|-----------|--------|
| 1 | Luis       | Popp      | 6900   |
| 2 | James      | Landry    | 2400   |
| 3 | James      | Marlow    | 2500   |
| 4 | Curtis     | Davies    | 3100   |
| 5 | Charles    | Johnson   | 6200   |
| 6 | Alexis     | Bull      | 4100   |
| 7 | Douglas    | Grant     | 2600   |

|   | FIRST_NAME | LAST_NAME | SALARY |
|---|------------|-----------|--------|
| 1 | Shanta     | Vollman   | 6500   |
| 2 | Michael    | Rogers    | 2900   |
| 3 | Charles    | Johnson   | 6200   |
| 4 | Martha     | Sullivan  | 2500   |
| 5 | Michael    | Hartstein | 13000  |



```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE First_name LIKE '%ha%';
```

## SELECT- IS NULL

- Verifica quais campos estão com valores nulos
- No exemplo ao lado são listados os dados dos funcionários que não tem gerente associado, pois o campo de identificação do gerente está nulo.

```
SELECT First_name, Last_name,  
        Salary, Manager_id  
FROM Employees  
WHERE Manager_id IS NULL;
```

```
SELECT city, state_province  
FROM Locations  
WHERE state_province IS NULL;
```

- No exemplo ao lado são listados os dados de localizações que não tem estado informado.

## SELECT– Operadores SQL Negativos

| Operador                | Significado                                   |
|-------------------------|-----------------------------------------------|
| NOT BETWEEN ... AND ... | Valores fora da faixa definida                |
| NOT IN (lista)          | Valores que não constam da lista              |
| NOT LIKE                | Valores que não são iguais a cadeia informada |
| IS NOT NULL             | Tudo que não é nulo                           |

## **SELECT– Operadores SQL Negativos**

```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE Salary NOT BETWEEN 10000 AND 20000;
```

```
SELECT First_name, Last_name, Salary, Manager_id  
FROM Employees  
WHERE Manager_id NOT IN (101, 145, 149);
```

```
SELECT First_name, Last_name, Salary, Manager_id  
FROM Employees  
WHERE first_name NOT LIKE '%d' ORDER BY first_name;
```

```
SELECT city, state_province  
FROM Locations  
WHERE state_province IS NOT NULL;
```

## SELECT– Múltiplas condições

- Os operadores **AND** e **OR** devem ser usados para fazer composições de expressões lógicas.
- **AND**: espera que ambas as condições sejam verdadeiras;
- **OR**: espera que uma das condições seja verdadeira.
- Nos dois exemplos seguintes as condições são as mesmas, mas o operador é diferente.
- Veja como o resultado é totalmente diferente.

# SELECT– Múltiplas condições - Exemplos

```
SELECT First_name, Last_name, Salary, Job_ID  
FROM Employees  
WHERE Salary BETWEEN 2000 AND 3000  
      AND Job_id = 'ST_CLERK';
```

Mais  
restritivo

- ❑ O código acima recupera somente os dados de funcionários que atendam ambas as condições simultaneamente

```
SELECT First_name, Last_name, Salary, Job_ID  
FROM Employees  
WHERE Salary BETWEEN 2000 AND 3000  
      OR Job_id = 'ST_CLERK';
```

Menos  
restritivo

- ❑ O código acima recupera os dados de funcionários que atendam a pelo menos uma das condições

## SELECT– Múltiplas condições - Exemplos

- ❑ Você pode combinar **AND** e **OR** na mesma expressão lógica.
- ❑ Quando **AND** e **OR** aparecer na mesma cláusula **WHERE**, todos os **ANDs** serão feitos primeiros e posteriormente todos os “**ORs**” serão feitos.

```
SELECT First_name, Last_name, Salary, Job_ID
FROM Employees
WHERE Salary > 8000 AND
      Job_id = 'ST_CLERK' OR Job_id = 'ST_MAN'
```

Esta consulta só traz registros que satisfazem a segunda condição porque não nenhum que satisfaz a primeira.

|   | FIRST_NAME | LAST_NAME | SALARY | JOB_ID |
|---|------------|-----------|--------|--------|
| 1 | Matthew    | Weiss     | 8000   | ST MAN |
| 2 | Adam       | Fripp     | 8200   | ST MAN |
| 3 | Pavam      | Kaufling  | 7900   | ST MAN |
| 4 | Shanta     | Vollman   | 6500   | ST MAN |
| 5 | Kevin      | Mourgos   | 5800   | ST MAN |



## SELECT– Múltiplas condições - Exemplos

```
SELECT First_name, Last_name, Salary, Job_ID
FROM Employees
WHERE Salary > 8000 AND
      (Job_id = 'ST_CLERK' OR Job_id = 'ST_MAN')
```

- Os parênteses definem a prioridade de avaliação da expressão lógica.
- Neste caso, o operador **OR** é avaliado antes do **AND**.

| FIRST_NAME | LAST_NAME | SALARY | JOB_ID |
|------------|-----------|--------|--------|
| 1 Adam     | Fripp     | 8200   | ST MAN |

## SELECT– Cláusula ORDER BY

- ❑ Para definir a ordem das linhas retornadas de uma consulta utilizamos a cláusula **ORDER BY**.
- ❑ Ele sempre deve ser a última parte da declaração **SELECT**.
- ❑ Acompanhando o **ORDER BY** devemos indicar o campo pelo qual desejamos ordenar.

```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE Salary BETWEEN 10000 AND 20000  
ORDER BY Last_name;
```

- ❑ A ordenação por padrão é ascendente, ou seja, do menor para o maior.

## SELECT– Cláusula ORDER BY

- Para recuperar os registros em ordem inversa, use a palavra DESC (decrescente) após o nome do campo.

```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE Salary BETWEEN 10000 AND 20000  
ORDER BY Last_name DESC;
```

- Também é possível ordenar mais de um campo, separando-os com vírgula.

```
SELECT First_name, Last_name, Salary  
FROM Employees  
WHERE Salary BETWEEN 10000 AND 20000  
ORDER BY salary DESC, Last_name ;
```

## **SELECT– Cláusula ORDER BY**

- ❑ Para ordenar por uma coluna, ela não precisa necessariamente estar declarada no **SELECT**.
- ❑ A cláusula **ORDER BY** é usada na pesquisa quando você quer mostrar as linhas em uma ordem específica. Sem a cláusula **ORDER BY** as linhas são retornadas na ordem conveniente para o ORACLE.
- ❑ Esse comando não altera a ordem dos dados que estão armazenados no Banco de Dados.

# Funções

- ❑ As Funções auxiliam na manipulação dos dados.
- ❑ Elas aceitam um ou mais argumentos e retornam um valor.
- ❑ Um argumento pode ser uma variável/valor ou coluna/campo.
- ❑ A sintaxe básica de uma função é a seguinte:

**função\_nome** (**argumento1, argumento2, ...**)

- ❑ Funções podem ser usadas para:
  - ❑ Cálculos sobre datas;
  - ❑ Modificar valores de itens individuais;
  - ❑ Manipular saída para grupos de linhas;
  - ❑ Alterar formatos de datas para mostrá-los.

# Uso de campos com valores Nulos

- Nulo significa que o valor está indisponível ou desconhecido, ou seja, não é o mesmo que 0 (zero), pois este é um número. Nulo é ausência de valor.
- Quando utilizamos uma coluna com valor nulo em uma expressão, mesmos que os outros valores envolvidos não sejam nulos, o resultado será nulo. Veja o exemplo abaixo.

```
SELECT First_name,  
Last_name,  
Salary * 12 + Commission_pct  
"Salário Anual"  
FROM Employees;
```

|   | ⚡ FIRST_NAME | ⚡ LAST_NAME | ⚡ Salário Anual |
|---|--------------|-------------|-----------------|
| 1 | Steven       | King        | (null)          |
| 2 | Neena        | Kochhar     | (null)          |
| 3 | Lex          | De Haan     | (null)          |
| 4 | Alexander    | Hunold      | (null)          |

# Função NVL

- Para realizar o cálculo é necessário converter os valores nulos para numéricos, para isto podemos utilizar a função NVL que converte valores nulos para não nulos. Isto se aplica a qualquer tipo de dado. Sua sintaxe é:

**NVL**(coluna\_ou\_campo\_nulo, valor\_não\_nulo)

- Veja o exemplo:

```
SELECT First_name, Last_name,  
Salary * 12 + NVL(Commission_pct,  
0) "Salário Anual"  
FROM Employees;
```

|   | ⚡ FIRST_NAME | ⚡ LAST_NAME | ⚡ Salário Anual |
|---|--------------|-------------|-----------------|
| 1 | Steven       | King        | 288000          |
| 2 | Neena        | Kochhar     | 204000          |
| 3 | Lex          | De Haan     | 204000          |
| 4 | Alexander    | Hunold      | 108000          |

# Função NVL2

- Esta função converte valores nulos para não nulos, mas permite retorna diferentes valores dependendo do campo estar ou não com nulo. Isto se aplica a qualquer tipo de dado.

Sua sintaxe é:

**NVL2**(coluna\_ou\_campo\_nulo,  
valor\_se\_não\_nulo,  
valor\_se\_for\_nulo)

|   | FINAL                            |
|---|----------------------------------|
| 1 | Roma                             |
| 2 | Venice                           |
| 3 | Tokyo - Tokyo Prefecture         |
| 4 | Hiroshima                        |
| 5 | Southlake - Texas                |
| 6 | South San Francisco - California |

- Veja o exemplo:

```
SELECT NVL2(state_province, city || ' - ' || state_province, city)
final
FROM Locations;
```



# Funções Alfanuméricas

- Estas funções aceitam dados alfanuméricos e podem retornar valores alfanuméricos ou numéricos. Trataremos das seguintes:
- LOWER/UPPER/INITCAP;
- LPAD/RPAD;
- REPLACE/TRANSLATE;
- SUBSTR;
- TRIM;
- INSTR;
- LENGTH.

# Função LOWER

- Esta função converte caracteres alfanuméricos para minúsculo. Sua sintaxe é:

**LOWER**(campo\_ou\_cadeia)

- Veja o exemplo:

```
SELECT LOWER(department_name),  
           LOWER('BANCO DE DADOS')  
FROM departments;
```

```
SELECT first_name, salary  
FROM employees  
WHERE LOWER(first_name) = 'steven';
```

# Função UPPER

- Esta função converte caracteres alfanuméricos para maiúsculo. Sua sintaxe é:

**UPPER**(campo\_ou\_cadeia)

- Veja o exemplo:

```
SELECT UPPER(first_name),      UPPER(last_name)  
FROM employees;
```

```
SELECT first_name, salary  
FROM employees  
WHERE UPPER(first_name) = 'STEVEN';
```

# Função INITCAP

- Esta função converte o primeiro caracter alfanumérico de cada palavra para maiúsculo. Sua sintaxe é:

**INITCAP**(campo\_ou\_cadeia)

- Veja o exemplo:

```
SELECT country_name, INITCAP(country_name) "pais"  
FROM Countries  
WHERE country_id='US';
```

|   | COUNTRY_NAME             | pais                     |
|---|--------------------------|--------------------------|
| 1 | United States of America | United States Of America |



# Função LPAD

- Esta função preenche a esquerda da **cadeia** de caracteres com uma quantidade **n** do **caracter** especificado. Sua sintaxe é:

**LPAD(campo\_ou\_cadeia, n, 'caracter')**

- Veja o exemplo:

```
SELECT LPAD(Department_name,20,'*'),  
      LPAD(Department_name,20),  
      Department_name,  
      LPAD(Department_name,20,':')  
FROM Departments;
```

# Função RPAD

- Esta função preenche a direita da **cadeia** de caracteres com uma quantidade **n** do **caracter** especificado. Sua sintaxe é:

**RPAD(campo\_ou\_cadeia, n, 'caracter')**

- Veja o exemplo:

```
SELECT RPAD(Department_name,20,'*'),  
       RPAD(Department_name,20),  
       Department_name,  
       RPAD(Department_name,20,',')  
FROM Departments;
```

# Função RPAD

- Esta função preenche a direita da **cadeia** de caracteres com uma quantidade **n** do **caracter** especificado. Sua sintaxe é:

- **RPAD(campo\_ou\_cadeia, n, 'caracter')**  
Veja o exemplo:

```
SELECT RPAD(Department_name,20,'*'),  
        RPAD(Department_name,20),  
        RPAD(Department_name,20,',')  
FROM Departments;
```

# Função TRANSLATE

- Substitui o caracter ou expressão “de”, para o caracter “para”. Se o parâmetro para for omitido o mesmo será removido. Sua sintaxe é:

**TRANSLATE(campo, de, para)**

- Veja o exemplo:

```
SELECT first_name,  
        TRANSLATE(  
            UPPER(first_name),'L','X')  
FROM Employees;
```

|   | ⚡ FIRST_NAME | ⚡ TRANSLATE(UPPER... |
|---|--------------|----------------------|
| 1 | Ellen        | EXXEN                |
| 2 | Sundar       | SUNDAR               |
| 3 | Mozhe        | MOZHE                |
| 4 | David        | DAVID                |
| 5 | Hermann      | HERMANN              |
| 6 | Shelli       | SHEXXI               |
| 7 | Amit         | AMIT                 |
| 8 | Elizabeth    | EXIZABETH            |



# Função REPLACE

- Substitui a expressão “de”, para a expressão “para”. Se o parâmetro para for omitido o mesmo será removido. Sua sintaxe é:

**REPLACE(campo, de, para)**

- Veja o exemplo:

```
SELECT Job_Title,  
       REPLACE(Job_Title,'Finance Manager','Gerente Financeiro'),  
       REPLACE(Job_Title,'nt','XX'),  
       REPLACE(Job_Title,'Administration')  
FROM Jobs;
```

# Função REPLACE X TRANSLATE

```
SELECT Job_Title,  
        REPLACE(Job_Title,'nt','XXSS'),  
        TRANSLATE(Job_Title,'nt','XSS')  
FROM Jobs;
```

| ⚡ JOB_TITLE                     | ⚡ REPLACE(JOB_TITLE,'NT','XXSS') | ⚡ TRANSLATE(JOB_TITLE,'NT','XSS') |
|---------------------------------|----------------------------------|-----------------------------------|
| 1 President                     | PresideXXSS                      | PresideXS                         |
| 2 Administration Vice President | Administration Vice PresideXXSS  | AdmiXisSraSioX Vice PresideXS     |
| 3 Administration Assistant      | Administration AssistaXXSS       | AdmiXisSraSioX AssisSaXS          |
| 4 Finance Manager               | Finance Manager                  | FiXaXce MaXager                   |
| 5 Accountant                    | AccouXXSSaXXSS                   | AccouXSaXS                        |

A Função REPLACE substitui uma expressão por outra e a TRANSLATE substitui caractere por caractere.

# Função SUBSTR

- Retorna um pedaço da cadeia de caracteres que vai do **início** até a quantidade definida em **comprimento**. Sua sintaxe é:

**SUBSTR(campo, inicio, comprimento)**

- Veja o exemplo:

```
SELECT first_name || ' ' || last_name nome,  
        SUBSTR(first_name || ' ' || last_name,2,3) particionado  
FROM employees;
```

```
SELECT city  
FROM locations  
WHERE SUBSTR(city,0,5) = 'South';
```

# Função TRIM

- Remove o caracter indicado, do início(leading), final (trailing) ou ambos (both), do conteúdo do campo. Sua sintaxe é:

**TRIM(LEADING** *caracter* **FROM** *campo*)

**TRIM(TRAILING** *caracter* **FROM** *campo*)

**TRIM(BOTH** *caracter* **FROM** *campo*)

- Veja o exemplo:

```
SELECT city, TRIM(LEADING 'S' FROM city) "removido"  
FROM locations WHERE city LIKE 'S%';
```

```
SELECT first_name, TRIM(TRAILING 's' FROM first_name) "removido"  
FROM employees WHERE first_name LIKE '%s';
```

```
SELECT first_name, TRIM(BOTH 'e' FROM LOWER(first_name)) "removido"  
FROM employees;
```

# Função INSTR

- Procura uma **cadeia** de caracteres no conteúdo do **campo**, retornando a posição onde começa a cadeia se esta existir. Sua sintaxe é:
- **INSTR(campo/valor, cadeia)** – retorna a primeira ocorrência  
**INSTR(campo/valor, cadeia, inicio, ocorrência)** – retorna a n-ésima ocorrência.
- Veja o exemplo:

```
SELECT first_name, INSTR(first_name,'l') "posicao"  
FROM employees;
```

```
SELECT first_name, INSTR(first_name,'n', 1, 2) "posicao"  
FROM employees;
```

```
SELECT first_name, INSTR(first_name,'n', -1, 1) "posicao"  
FROM employees;
```

# Função LENGTH

- Retorna o comprimento/tamanho da cadeia de caracteres/campo. Sua sintaxe é:

**LENGTH**(campo/valor)

```
SELECT first_name,  
       LENGTH(first_name) "tamanho",  
       LENGTH(first_name || ' ' || last_name) "tam_completo"  
FROM employees;
```

|   | FIRST_NAME | tamanho | tam_completo |
|---|------------|---------|--------------|
| 1 | Ellen      | 5       | 10           |
| 2 | Sundar     | 6       | 11           |
| 3 | Mozhe      | 5       | 14           |
| 4 | David      | 5       | 12           |

## Concatenando Funções

- ❑ Funções de linhas únicas podem ser concatenadas para várias finalidades.

- ❑ Elas serão avaliadas de dentro para fora. Exemplo:

**X(D(A(B(C(argumento)))))**

ordem lógica de execução **C**, **B**, **A**, **D** e **X**

- ❑ Suponhamos que você queira encontrar o número de vezes que um determinado caracter ocorre em uma linha. Como você faria isso?
- ❑ Você pode aninhar as funções LENGTH e TRANSLATE para realizar essa pesquisa.

# Concatenando Funções

```
SELECT Job_Title "titulo",  
       LENGTH(Job_Title) "tamanho",  
       LENGTH(Job_Title) -  
       LENGTH(TRANSLATE(UPPER(Job_Title),'AS','A')) "Qtde de  
S"
```

FROM Jobs;

Remove a letra "S", pois  
não tem letra para  
substituí-la

Converte para  
maiúsculo

Retorna o tamanho  
cadeia sem o "S"

|   | titulo                        | tamanho | Qtde de S |
|---|-------------------------------|---------|-----------|
| 1 | President                     | 9       | 1         |
| 2 | Administration Vice President | 29      | 2         |
| 3 | Administration Assistant      | 24      | 4         |
| 4 | Finance Manager               | 15      | 0         |
| 5 | Accountant                    | 10      | 0         |



# Concatenando Funções

- ❑ Vamos entender o que aconteceu para realizar essa pesquisa.
  - ❑ Usa-se o LENGTH para identificar o número de caracteres da linha e a função TRANSLATE para eliminar todas as ocorrências de S.
  - ❑ Note que A é modificado para A, e S não corresponde a nenhum caractere para ser alterado. Como S não tem nenhum caractere correspondente ele é removido da linha.
  - ❑ Agora subtrai o tamanho da linha retirado os S's do tamanho da linha original. (com os S's) `Length(Job_Title)-Length(Translate(Upper(Job_Title),'AS','A'))`
  - ❑ O resultado é o número de ocorrências do caracter 'S' na linha.

# Funções Numéricas

- ❑ ROUND/FLOOR/CEIL
- ❑ TRUNC
- ❑ POWER
- ❑ SQRT
- ❑ MOD
- ❑ SUM
- ❑ AVG
- ❑ COUNT
- ❑ MAX
- ❑ MIN

# Função ROUND

- Arredonda o valor com “n” casas decimais. Se “n” for omitido não serão definidas casas decimais. Valores iguais ou superiores a 5, serão arredondados para cima e inferiores para baixo. Sua sintaxe é:

**ROUND(campo/valor, n)**

```
SELECT  
ROUND(45.923,1), ROUND(45.923), ROUND(45.323,1),  
ROUND(44.323,-1), ROUND(45.323,-1), ROUND(155.323,-2),  
Salary/32, ROUND(Salary/32,2)  
FROM Employees;
```

|   | ROUND(45.923,1) | ROUND(45.923) | ROUND(45.323,1) | ROUND(44.323,-1) | ROUND(45.323,-1) | ROUND(155.323,-2) | SALARY/32 | ROUND(SALARY/32,2) |
|---|-----------------|---------------|-----------------|------------------|------------------|-------------------|-----------|--------------------|
| 1 | 45,9            | 46            | 45,3            | 40               | 50               | 200               | 750       | 750                |
| 2 | 45,9            | 46            | 45,3            | 40               | 50               | 200               | 531,25    | 531,25             |
| 3 | 45,9            | 46            | 45,3            | 40               | 50               | 200               | 531,25    | 531,25             |

# Função FLOOR

- ❑ Pega somente a parte inteira do número, retirando a parte decimal e não faz arredondamento para cima, somente para baixo. Sua sintaxe é:

## FLOOR(campo/valor)

```
SELECT FLOOR(45.923), FLOOR(45.323),  
Salary/1.5, FLOOR(Salary/1.5) FROM Employees;
```

[illegible]

# Função CEIL

- ❑ Pega somente a parte inteira do número, retirando a parte decimal e não faz arredondamento para baixo, somente para cima. Sua sintaxe é:

## CEIL(campo/valor)

```
SELECT CEIL(45.923), CEIL(45.323),  
Salary/1.5, CEIL(Salary/1.5) FROM  
Employees;
```

[illegible]

# Função TRUNC

- Trunca o valor com “n” casas decimais. Se “n” for omitido não serão definidas casas decimais. Sua sintaxe é:

**TRUNC(campo/valor, n)**

```
SELECT
TRUNC(45.923,1), TRUNC(45.923), TRUNC(45.323,1),
TRUNC(44.323,-1), TRUNC(46.323,-1),
TRUNC(155.323,-2), Salary/32, TRUNC(Salary/32,2)
FROM Employees;
```

|   | TRUNC(45.923,1) | TRUNC(45.923) | TRUNC(45.323,1) | TRUNC(44.323,-1) | TRUNC(46.323,-1) | TRUNC(155.323,-2) | SALARY/32 | TRUNC(SALARY/32,2) |
|---|-----------------|---------------|-----------------|------------------|------------------|-------------------|-----------|--------------------|
| 1 | 45,9            | 45            | 45,3            | 40               | 40               | 100               | 750       | 750                |
| 2 | 45,9            | 45            | 45,3            | 40               | 40               | 100               | 531,25    | 531,25             |
| 3 | 45,9            | 45            | 45,3            | 40               | 40               | 100               | 531,25    | 531,25             |

# Função POWER

- Eleva um campo ou valor na potência definida em “n”. Sua sintaxe é:

**POWER**(campo/valor, n)

```
SELECT salary, POWER(salary,2),  
        POWER(43.5,2), POWER(50,2)  
FROM Employees  
WHERE department_id = 30;
```

```
SELECT min_salary, POWER(min_salary,2)  
FROM Jobs  
WHERE min_salary <= 3000;
```

## Função SQRT

- Retorna a raiz quadrada do campo ou valor informado. Sua sintaxe é:

**SQRT**(campo/valor)

```
SELECT salary, SQRT(Salary), SQRT(43.5), SQRT(50)  
FROM Employees  
WHERE department_id = 30;
```

```
SELECT salary, ROUND(SQRT(salary),2),  
          ROUND(SQRT(43.5),3), ROUND(SQRT(50),2)  
FROM Employees  
WHERE Department_id = 30;
```



# Função MOD

- Retorna o resto da divisão do campo ou valor informado pelo valor de **n**. Sua sintaxe é:

**MOD**(campo/valor, n)

```
SELECT Salary,  
        Commission_pct * 33,  
        MOD(Salary, Commission_pct * 33),  
        MOD(100,40)  
FROM Employees  
WHERE Commission_pct <> 0;
```

# Funções

## Agregadoras

- A sintaxe geral é:

**funcao(N)**

| Função | Descrição                              |
|--------|----------------------------------------|
| SUM    | Retorna a soma de N                    |
| AVG    | Retorna a média aritmética de N        |
| COUNT  | Retorna o número de linhas da consulta |
| MAX    | Retorna o valor máximo de N            |
| MIN    | Retorna o valor mínimo de N            |

```
SELECT SUM(Salary), AVG(Salary), COUNT(*),  
       MAX(Salary), MIN(Salary)  
FROM Employees;
```

|   | SUM(SALARY) | AVG(SALARY)                               | COUNT(*) | MAX(SALARY) | MIN(SALARY) |
|---|-------------|-------------------------------------------|----------|-------------|-------------|
| 1 | 691416      | 6461,831775700934579439252336448598130841 | 107      | 24000       | 2100        |

```
SELECT COUNT(*),  
       COUNT(manager_id)  
FROM employees  
WHERE Salary > 10000;
```

|   | COUNT(*) | COUNT(MANAGER_ID) |
|---|----------|-------------------|
| 1 | 15       | 14                |

# Funções Agregadoras

```
SELECT COUNT(DISTINCT department_id)  
FROM employees;
```

| COUNT(DISTINCT DEPARTMENT_ID) |    |
|-------------------------------|----|
| 1                             | 11 |

```
SELECT SUM(Salary)  
FROM employees  
WHERE Department_id = 10;
```

| SUM(SALARY) |      |
|-------------|------|
| 1           | 4400 |

## Função Para Conversão

- ❑ **TO\_CHAR:** especifica que a data está sendo convertida para um novo formato na saída.
- ❑ É frequentemente usada para modificar um formato de data padrão para um formato alternativo.

```
SELECT hire_date, TO_CHAR(SYSDATE, 'DD month YYYY'),  
        TO_CHAR(hire_date,'day DD month YYYY') "extenso"  
FROM employees;
```

|   | ⚡ HIRE_DATE | ⚡ TO_CHAR(SYSDATE,'DDMONTHYYYY') | ⚡ extenso                     |
|---|-------------|----------------------------------|-------------------------------|
| 1 | 17/06/03    | 18 março 2018                    | terça-feira 17 junho 2003     |
| 2 | 21/09/05    | 18 março 2018                    | quarta-feira 21 setembro 2005 |
| 3 | 13/01/01    | 18 março 2018                    | sábado 13 janeiro 2001        |

```
SELECT TO_CHAR(SYSDATE, 'HH:MI:SS')  
FROM dual;
```

|   | ⚡ TO_CHAR(SYSDATE,'HH:MI:SS') |
|---|-------------------------------|
| 1 | 10:09:52                      |

# Funções Para Conversão

## Formatos Alfanuméricos

| Máscara               | Significado                                                             |
|-----------------------|-------------------------------------------------------------------------|
| YYYY ou YY ou YY ou Y | Últimos 4,3,2 ou 1 dígito(s) do ano                                     |
| Q                     | Um quarto do Ano                                                        |
| MM                    | Mês                                                                     |
| MONTH                 | Nome do mês, espaçamento com brancos do tamanho de 9 caracteres         |
| MON                   | Nome do mês com 3 letras abreviadas                                     |
| WW                    | Semana do ano                                                           |
| DDD ou DD             | Dia do ano ou dia do mês                                                |
| DAY                   | Nome do dia da semana, espaçado com brancos com 9 caracteres de tamanho |
| DY                    | nome do dia da semana com 3 letras abreviadas                           |

**(continua...)**

# Funções Para Conversão

## Formatos Alfanuméricos (continuação da tabela)

| Máscara    | Significado                           |
|------------|---------------------------------------|
| AM ou PM   | Indicador meridiano                   |
| HH ou HH12 | Horas do dia (0-12)                   |
| HH24       | Horas do dia (0-23)                   |
| MI         | Minutos                               |
| SS         | Segundos                              |
| SSSSS      | Segundos passado meia-noite (0-86399) |

## Funções Para Conversão

### Mais exemplos:

```
SELECT first_name, hire_date, TO_CHAR(hire_date, 'DD/MM/YYYY HH24:MI:SS')  
FROM Employees;
```

```
SELECT first_name, hire_date, TO_CHAR(hire_date, 'DDD DD/MM/YYYY -  
HH24:MI:SSSSS') FROM Employees;
```

```
SELECT first_name, hire_date, TO_CHAR(hire_date, 'Q DD/MONTH/YYYY') FROM  
Employees;
```

```
SELECT first_name, hire_date, TO_CHAR(hire_date, 'WW - DAY  
DD/MONTH/YYYY') FROM Employees;
```

```
SELECT first_name, hire_date, TO_CHAR(SYSDATE, 'DD/MM/YYYY'),  
TO_CHAR(hire_date, 'DD/MM/YYYY'), TRUNC(SYSDATE), TRUNC(hire_date)  
FROM Employees;
```

# Funções Para Conversão

- ❑ **TO\_DATE:** converte para data (date) uma expressão alfanumérica que represente uma data.

**SELECT**

**TO\_DATE('Janeiro 15, 1989, 11:00', 'Month dd, YYYY, HH:MI')**  
"passado",

**TO\_DATE('19 março 2017', 'dd month YYYY')** "hoje"

**FROM** dual;

|   | passado  | hoje     |
|---|----------|----------|
| 1 | 15/01/89 | 19/03/17 |

<https://docs.oracle.com/database/121/SQLRF/toc.htm>



## Funções de Data (Date)

- **Extract:** retorna uma das informações da data, devendo ser indicada qual desejamos. Sua sintaxe é:

**EXTRACT**(**informacao FROM data**)

- Por exemplo:

```
SELECT EXTRACT(YEAR FROM hire_date)  
FROM employees;
```

```
SELECT hire_date, EXTRACT(month FROM hire_date)  
FROM employees;
```

- **Sysdate:** retorna a hora e hora do sistema.

# DECODE - Função de Decisão

- Esta função é como uma estrutura condicional da linguagem de programação, podemos considerá-la equivalente ao switch-case.
- A sintaxe básica: 

```
DECODE (campo/valor,  
        procurado1, resultado1,  
        procurado2, resultado2,  
        ...  
        procuradoN, resultadoN,  
        padrão  
    )
```

# DECODE - Função de Decisão

Ele deve ter no mínimo 3 argumentos.

- **Campo/valor:** o nome do campo ou expressão a ser avaliada.
- **Procurado1:** o primeiro valor para ser testado
- **Resultado1:** o valor para ser retornado se o **Procurado1** for encontrado.
- **Procurado1** e **Resultado1** podem ser repetidos quantas vezes forem necessários, ficariam assim:
  - **Procurado2, Resultado2, Procurado3, Resultado3, ...**
- **Padrão:** o valor a ser retornado se nenhum procurado for encontrado.
- **OBS:** campo/valor pode ser vários tipos de dados

# Função de DECODE

- O seguinte exemplo decodifica os cargos dos tipos IT\_PROG e FI\_ACCOUNT unicamente. Os outros cargos serão padrão alterados para INDEFINIDO.

```
SELECT first_name, job_id,  
        DECODE( job_id, 'IT_PROG','PROGRAMADOR',  
                'FI_ACCOUNT','CONTADOR', 'INDEFINIDO')  
decode_cargo  
FROM Employees;
```

|   | ⌘ FIRST_NAME | ⌘ JOB_ID | ⌘ DECODE_CARGO |
|---|--------------|----------|----------------|
| 1 | Ellen        | SA_REP   | INDEFINIDO     |
| 2 | Sundar       | SA_REP   | INDEFINIDO     |
| 3 | Mozhe        | ST_CLERK | INDEFINIDO     |
| 4 | David        | IT_PROG  | PROGRAMADOR    |
| 5 | Hermann      | PR_REP   | INDEFINIDO     |

# Função de DECODE

- Para mostrar a gratificação percentual dependendo da região:

```
SELECT region_id,  
        DECODE( region_id,  
        1, '15%',  
        2, '10%',  
        3, '8%',  
        '5%') bonus  
FROM Regions;
```

|   | REGION_ID | BONUS |
|---|-----------|-------|
| 1 | 1         | 15%   |
| 2 | 2         | 10%   |
| 3 | 3         | 8%    |
| 4 | 4         | 5%    |

# Função de DECODE

- Para mostrar o valor de reajuste somente dos salários iguais a 6000 ou 4200:

```
SELECT salary,  
        DECODE(salary,  
        6000, salary*5/100,  
        4200, salary*10/100,  
        salary) reajuste  
FROM employees;
```

|    | ⚡ SALARY | ⚡ REAJUSTE |
|----|----------|------------|
| 1  | 24000    | 24000      |
| 2  | 17000    | 17000      |
| 3  | 17000    | 17000      |
| 4  | 9000     | 9000       |
| 5  | 6000     | 300        |
| 6  | 4800     | 4800       |
| 7  | 4800     | 4800       |
| 8  | 4200     | 420        |
| 9  | 12008    | 12008      |
| 10 | 9000     | 9000       |
| 11 | 8200     | 8200       |

# Função de DECODE

- Para mostrar o valor de reajuste somente dos salários dos cargos com os códigos especificados:

```
SELECT first_name, job_id, salary,  
        DECODE(job_id,  
        'IT_PROG', salary * 1.1,  
        'FI_ACCOUNT', salary * 1.2,  
        'AD_VP', salary * 0.95,  
        salary) depte_cargo  
FROM Employees ;
```

|   | FIRST_NAME | JOB_ID  | SALARY | DEPTE_CARGO |
|---|------------|---------|--------|-------------|
| 1 | Steven     | AD_PRES | 24000  | 24000       |
| 2 | Neena      | AD_VP   | 17000  | 16150       |
| 3 | Lex        | AD_VP   | 17000  | 16150       |
| 4 | Alexander  | IT_PROG | 9000   | 9900        |
| 5 | Bruce      | IT_PROG | 6000   | 6600        |
| 6 | David      | IT_PROG | 4800   | 5280        |
| 7 | Valli      | IT_PROG | 4800   | 5280        |
| 8 | Diana      | IT_PROG | 4200   | 4620        |
| 9 | Nancy      | FI_MGR  | 12008  | 12008       |

# CASE - Expressão de Decisão

- Esta expressão também é como uma estrutura condicional da linguagem de programação, podemos considerá-la equivalente ao **if-else-if** ou **if-else**.
- A sintaxe básica:

**CASE**

```
    WHEN condicao1 THEN resultadoVerdadeiro1  
    [WHEN condicao2 THEN resultadoVerdadeiro2]  
    [ELSE resultadoFalso]
```

**END**

O que está entre colchetes é opcional.



# CASE - Expressão de Decisão

- **condição:** é uma expressão lógica que permite comparações utilizando operadores relacionais e lógicos.
- **resultadoVerdadeiroN:** o valor para ser retornado se a **condiçãoN** for verdadeira.
- **else** retorna o **resultadoFalso** se as **condições** (todas) forem falsas.

# CASE - Expressão de Decisão

```
SELECT first_name, job_id, salary,  
      CASE  
        WHEN salary < 5000 THEN 'AUMENTO'  
        WHEN salary > 10000 AND job_id='AD_VP' THEN 'VERIFICAR'  
        ELSE 'NÃO AUMENTAR'  
      END "classificação"  
FROM Employees;
```

|   | ⚡ FIRST_NAME | ⚡ JOB_ID | ⚡ SALARY | ⚡ classificação |  |
|---|--------------|----------|----------|-----------------|--|
| 1 | Steven       | AD_PRES  | 24000    | NÃO AUMENTAR    |  |
| 2 | Neena        | AD_VP    | 17000    | VERIFICAR       |  |
| 3 | Lex          | AD_VP    | 17000    | VERIFICAR       |  |
| 4 | Alexander    | IT_PROG  | 9000     | NÃO AUMENTAR    |  |
| 5 | Bruce        | IT_PROG  | 6000     | NÃO AUMENTAR    |  |
| 6 | David        | IT_PROG  | 4800     | AUMENTO         |  |

# CASE - Expressão de Decisão

- Para mostrar o valor dos novos salários somente para quem ganha abaixo de 5000, considerando diferentes aumentos dependendo da faixa salarial:

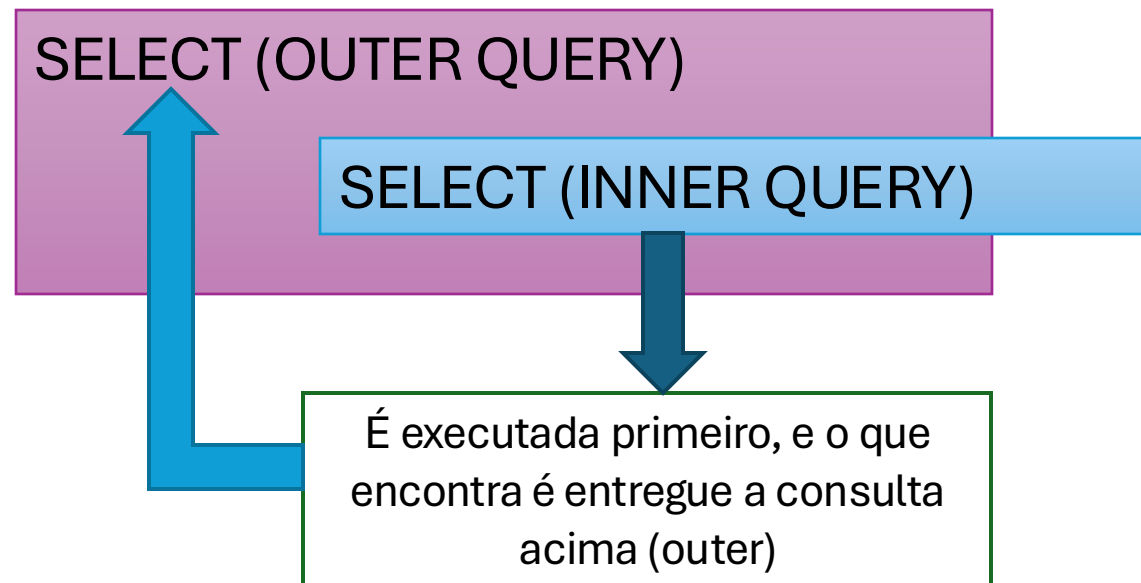
```
SELECT first_name, salary,  
       CASE  
         WHEN salary < 3000 THEN salary*1.1  
         WHEN salary < 5000 THEN salary*1.05  
         ELSE salary  
       END "novo salário"  
FROM Employees  
WHERE salary < 5000;
```

|   | FIRST_NAME | SALARY | novo salario |
|---|------------|--------|--------------|
| 1 | David      | 4800   | 5040         |
| 2 | Valli      | 4800   | 5040         |
| 3 | Diana      | 4200   | 4410         |
| 4 | Alexander  | 3100   | 3255         |
| 5 | Shelli     | 2900   | 3190         |
| 6 | Sigal      | 2800   | 3080         |
| - |            |        |              |

# SUBQUERY

# Subconsulta (Subquery)

- É um comando SELECT dentro de outro para atender a uma cláusula WHERE, podendo retornar uma ou mais linhas. Podemos dizer que é uma SELECT aninhada em outra.
- Ordem de execução:



# Subconsulta (Subquery)

A sintaxe básica:

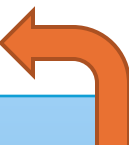
```
SELECT campos  
FROM tabela  
WHERE expressão operador  
      (SELECT campo FROM tabela);
```

# Subconsulta (Subquery)

- Exemplo 1:

```
SELECT last_name, salary  
FROM hr.employees  
WHERE salary >
```

Retorno de **um  
único valor**  
(single row)



```
(SELECT salary  
FROM hr.employees  
WHERE UPPER(last_name) = 'ABEL');
```

## **Sub-consulta (Subquery)**

- Exemplo 2: Listar os nomes e cargos dos funcionários com cargo igual ao do funcionário com número 141.



# Sub-consulta (Subquery)

- Exemplo 2: Listar os nomes e cargos dos funcionários com cargo igual ao do funcionário com número 141.

```
SELECT last_name, job_id  
FROM hr.employees  
WHERE job_id =
```

Consulta  
principal

```
(SELECT job_id  
FROM hr.employees  
WHERE employee_id = 141);
```

Subconsulta

## **Sub-consulta (Subquery)**

- Exemplo 3: Encontrar todos os funcionários que trabalham no mesmo departamento que “David Lee”

## Sub-consulta (Subquery)

- Exemplo 3: Encontrar todos os funcionários que trabalham no mesmo departamento que “David Lee”

```
SELECT first_name, last_name, job_id, department_id
FROM hr.employees
WHERE department_id=
    (SELECT department_id
     FROM hr.employees
     WHERE UPPER(first_name)= 'DAVID' AND
          UPPER(last_name)= 'LEE')
ORDER BY first_name;
```

## **Funções de Grupo + Single Row**

- Exemplo 4: Encontrar os funcionários que ganham acima da média salarial

## Funções de Grupo + Single Row

- Exemplo 4: Encontrar os funcionários que ganham acima da média salarial

```
SELECT first_name, last_name, job_id, department_id, salary  
FROM hr.employees  
WHERE salary > (SELECT AVG(salary) FROM hr.employees)  
ORDER BY first_name;
```

## **Sub-consulta Aninhada**

- Exemplo 5: Mostrar o primeiro e último nome, cargo, cód.do departamento e salário dos empregados cujo o salário seja superior ao maior salário no departamento 'SALES'.

## Sub-consulta Aninhada

- Exemplo 5: Mostrar o primeiro e último nome, cargo, cód.do departamento e salário dos empregados cujo o salário seja superior ao maior salário no departamento 'SALES'.

```
SELECT first_name, last_name, job_id, department_id, salary
FROM Employees WHERE salary >
    (SELECT MAX(salary)
     FROM hr.employees
     WHERE department_id=
        (SELECT department_id
         FROM hr.departments
         WHERE department_name='Sales')
    )
ORDER BY first_name;
```

# Sub-consulta para Múltiplas Linhas

## Operador EXISTS

- O operador EXISTS testa se um valor existe.(O **NOT EXISTS** garante que não existe). Se o valor existir será retornado Verdadeiro, se não existir será retornado Falso.
- Para encontrar os empregados que tem no mínimo uma pessoa subordinada a ele, faça:

```
SELECT employee_id, manager_id, first_name,  
        last_name, job_id, department_id, salary  
FROM hr.employees E  
WHERE EXISTS (SELECT employee_id  
                FROM hr.employees  
                WHERE E.manager_id = Employees.employee_id)  
ORDER BY employee_id;
```



# Sub-consulta para Múltiplas Linhas

## Operador NOT EXISTS

- Encontrar os empregados que não tem departamento relacionado na tabela **Departaments**:

```
SELECT employee_id, manager_id, first_name,  
        last_name, job_id, department_id, salary  
FROM hr.employees E  
WHERE NOT EXISTS (SELECT department_id  
        FROM hr.departments D  
        WHERE D.department_id= E.department_id) ;
```

# Sub-consulta para Múltiplas Linhas

## Operador NOT EXISTS

- Encontrar os departamentos que não tem funcionários relacionados na tabela **Employees**;

```
SELECT department_id, department_name  
FROM hr.departments D  
WHERE NOT EXISTS (SELECT 'X'  
                    FROM hr.employees E  
                    WHERE E.department_id = D.department_id)  
ORDER BY 2;
```



Ordena pela segunda coluna

# Sub-consulta para Múltiplas Linhas

## Operador IN

- Encontrar os funcionários que trabalham nos departamentos “Marketing” e “Administração”:

```
SELECT first_name, last_name,  
        job_id, department_id, salary  
FROM hr.employees  
WHERE department_id IN (SELECT department_id  
        FROM hr.departments  
        WHERE department_name IN  
        ('Marketing','Administration'))  
ORDER BY first_name;
```

# Sub-consulta para Múltiplas Linhas

## Operador ANY

- Compara o valor da esquerda com cada valor retornado pela *subquery*:

< ANY = menor que o maior valor  
> ANY = maior que o menor valor

```
SELECT employee_id, last_name, job_id, salary
FROM hr.employees
WHERE salary < ANY (SELECT salary
                        FROM hr.employees
                        WHERE job_id= 'IT_PROG')
AND job_id <> 'IT_PROG';
```

Neste caso esses valores são todos menores que 9000

# Sub-consulta para Múltiplas Linhas

## Operador ALL

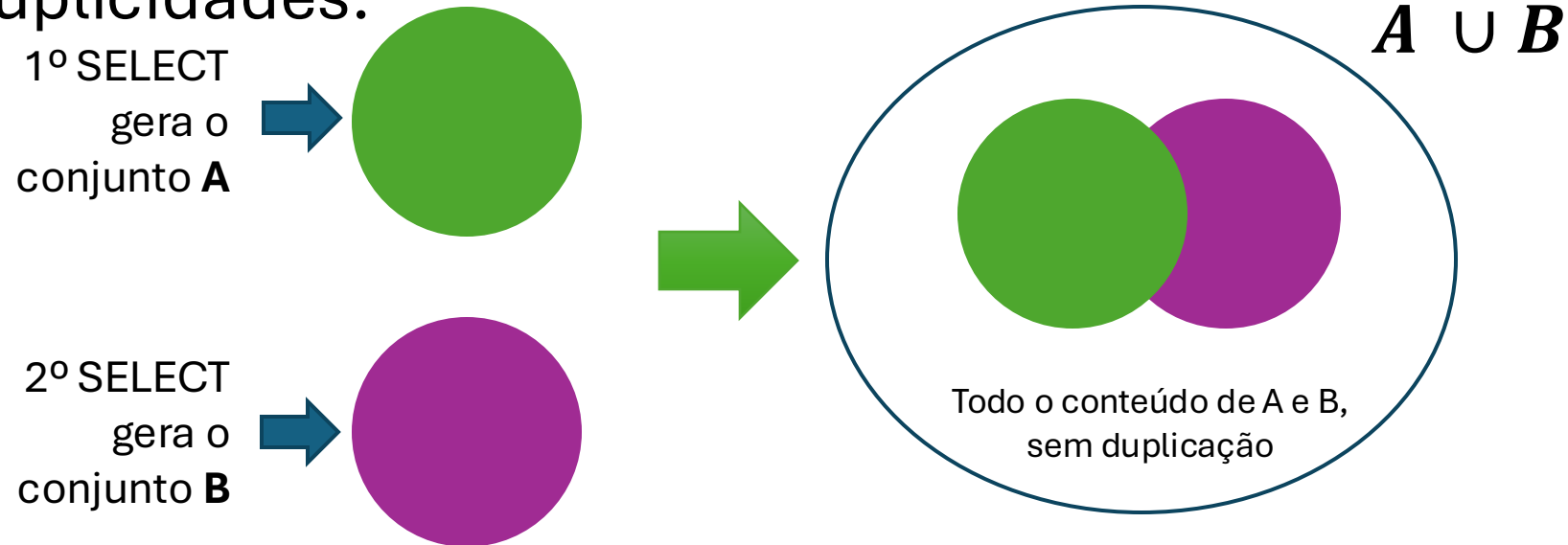
- Compara o valor da esquerda com o menor ou o maior valor retornado pela *subquery*:

```
SELECT employee_id, last_name, job_id, salary
FROM hr.employees
WHERE salary < ALL (SELECT salary
                        FROM hr.employees
                        WHERE job_id= 'IT_PROG')
AND job_id <> 'IT_PROG';
```

Neste caso esses valores são todos menores que 4200

# UNION

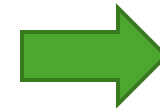
- Este operador permite combinar em um único retorno o resultado de dois SELECTs, retirando as duplicidades.



# UNION - Exemplo

- Neste exemplo, conseguimos saber as funções já desempenhadas pelos funcionários e as datas em que iniciaram.

```
SELECT employee_id, job_id  
FROM employees  
UNION  
SELECT employee_id, job_id  
FROM job_history;
```



|    | EMPLOYEE_ID | JOB_ID     |
|----|-------------|------------|
| 1  | 100         | AD_PRES    |
| 2  | 101         | AC_ACCOUNT |
| 3  | 101         | AC_MGR     |
| 4  | 101         | AD_VP      |
| 5  | 102         | AD_VP      |
| 6  | 102         | IT_PROG    |
| 7  | 103         | IT_PROG    |
| 8  | 104         | IT_PROG    |
| 9  | 105         | IT_PROG    |
| 10 | 106         | IT_PROG    |

# UNION - Exemplo

- Neste exemplo, conseguimos saber as funções já desempenhadas pelos funcionários.

```
SELECT employee_id, job_id, TO_DATE(null) "inicio"  
FROM employees  
UNION  
SELECT employee_id, job_id, start_date  
FROM job_history;
```

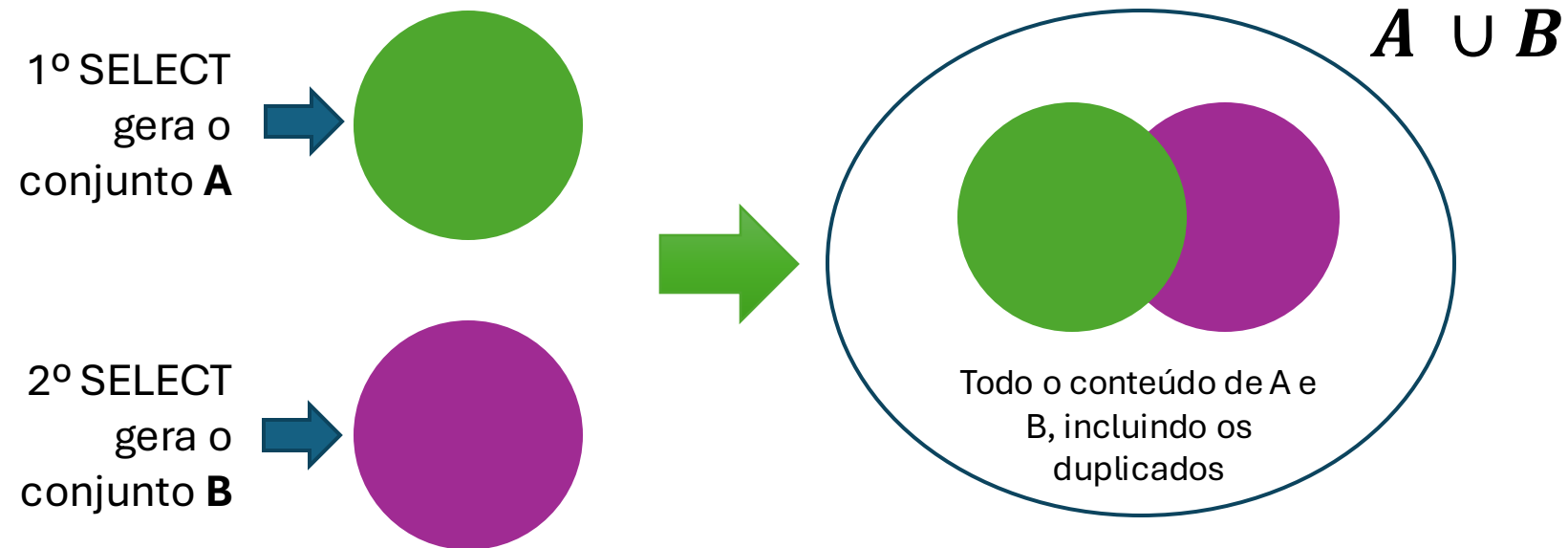


|   | EMPLOYEE_ID | JOB_ID     | inicio   |
|---|-------------|------------|----------|
| 1 | 100         | AD_PRES    | (null)   |
| 2 | 101         | AC_ACCOUNT | 21/09/97 |
| 3 | 101         | AC_MGR     | 28/10/01 |
| 4 | 101         | AD_VP      | (null)   |
| 5 | 102         | AD_VP      | (null)   |



# UNION ALL

- Este operador permite combinar em um único retorno o resultado de dois SELECTs, mantendo as duplicidades.



# UNION ALL - Exemplo

- Neste exemplo, vamos perceber que o funcionário com o id = 176 está retornando duplicado.

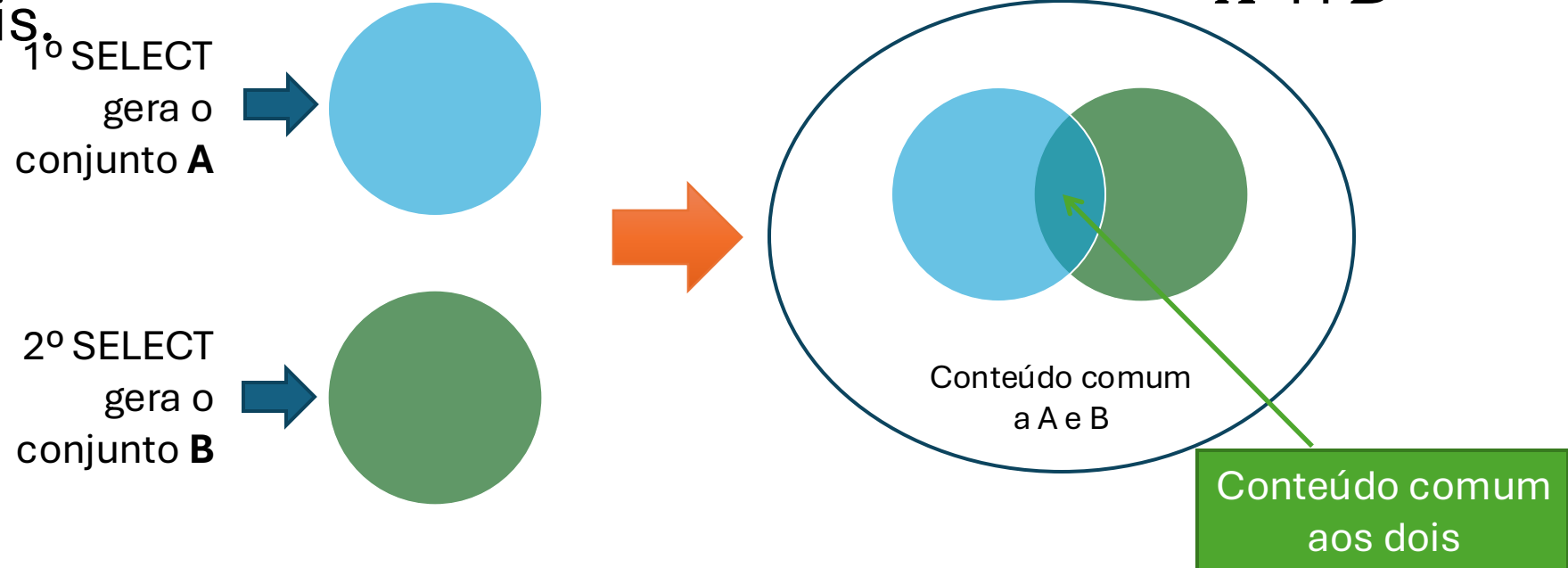
|    |     |        |
|----|-----|--------|
| 81 | 175 | SA_REP |
| 82 | 176 | SA_REP |
| 83 | 176 | SA_MAN |
| 84 | 176 | SA_REP |
| 85 | 177 | SA_REP |



```
SELECT employee_id, job_id  
FROM employees  
UNION ALL  
SELECT employee_id, job_id  
FROM job_history  
ORDER BY employee_id;
```

# INTERSECT

- Este operador permite obter em um único retorno os dados comuns nos dois SELECTs envolvidos, ou seja, retorna somente o que está presente nos dois.



# INTERSECT - Exemplo

- Neste exemplo, vamos perceber que são selecionados os dados que tem correspondência nas duas tabelas.

```
SELECT employee_id, job_id  
FROM employees  
INTERSECT  
SELECT employee_id, job_id  
FROM job_history  
ORDER BY employee_id;
```



|   | EMPLOYEE_ID | JOB_ID  |
|---|-------------|---------|
| 1 | 176         | SA_REP  |
| 2 | 200         | AD_ASST |

- Faça consultas nas tabelas separadamente e compare com este resultado.

# INTERSECT - Exemplo

Job\_history

|    | EMPLOYEE_ID | START_DATE | END_DATE | JOB_ID     | DEPARTMENT_ID |
|----|-------------|------------|----------|------------|---------------|
| 1  | 101         | 21/09/97   | 27/10/01 | AC_ACCOUNT | 110           |
| 2  | 101         | 28/10/01   | 15/03/05 | AC_MGR     | 110           |
| 3  | 102         | 13/01/01   | 24/07/06 | IT_PROG    | 60            |
| 4  | 114         | 24/03/06   | 31/12/07 | ST_CLERK   | 50            |
| 5  | 122         | 01/01/07   | 31/12/07 | ST_CLERK   | 50            |
| 6  | 176         | 24/03/06   | 31/12/06 | SA_REP     | 80            |
| 7  | 176         | 01/01/07   | 31/12/07 | SA_MAN     | 80            |
| 8  | 200         | 17/09/95   | 17/06/01 | AD_ASST    | 90            |
| 9  | 200         | 01/07/02   | 31/12/06 | AC_ACCOUNT | 90            |
| 10 | 201         | 17/02/04   | 19/12/07 | MK_REP     | 20            |

Employees

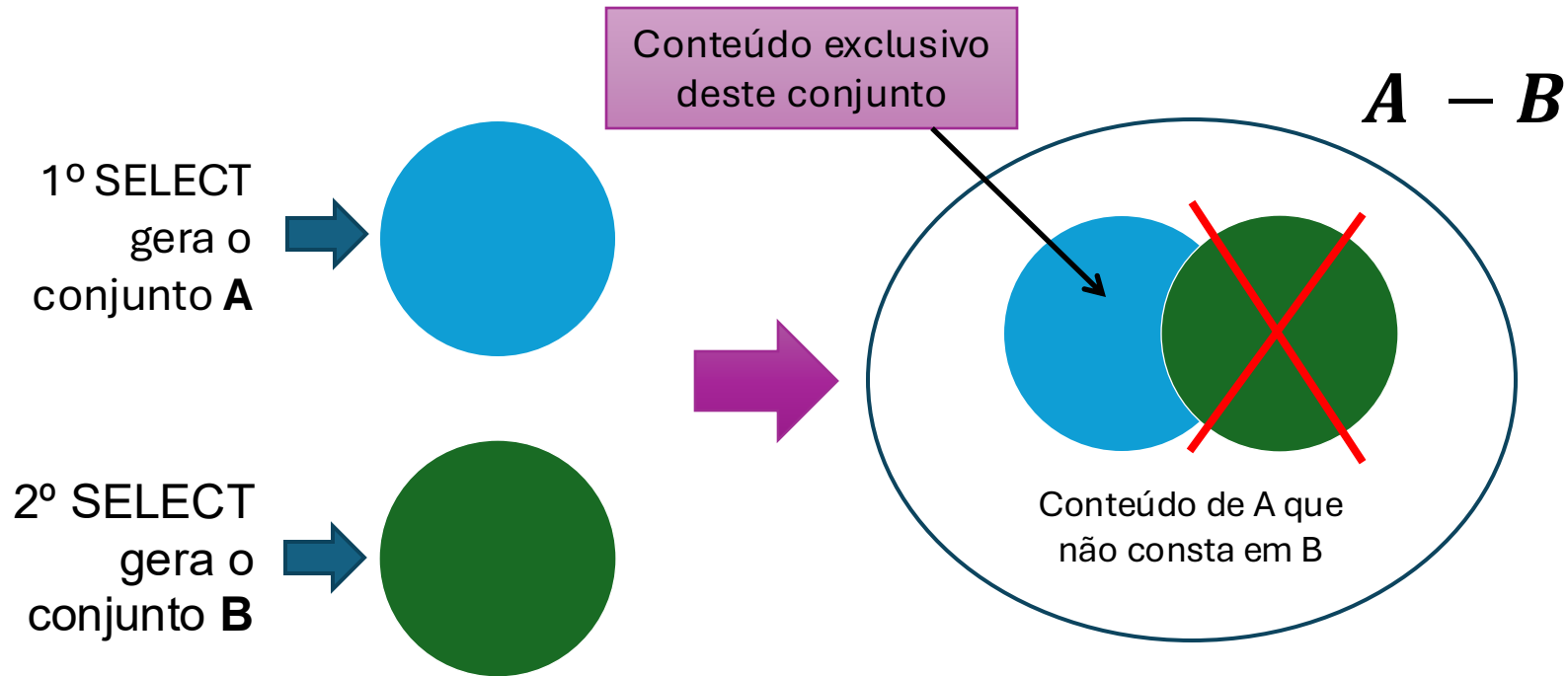
|    | EMPLOYEE_ID | JOB_ID |
|----|-------------|--------|
| 55 | 172         | SA_REP |
| 56 | 173         | SA_REP |
| 57 | 174         | SA_REP |
| 58 | 175         | SA_REP |
| 59 | 176         | SA_REP |
| 60 | 177         | SA_REP |
| 61 | 178         | SA_REP |

|   | EMPLOYEE_ID | JOB_ID     |
|---|-------------|------------|
| 1 | 206         | AC_ACCOUNT |
| 2 | 205         | AC_MGR     |
| 3 | 200         | AD_ASST    |
| 4 | 100         | AD PRES    |
| 5 | 101         | AD_VP      |
| 6 | 102         | AD_VP      |
| 7 | 109         | FI_ACCOUNT |
| 8 | ...         | ...        |

- Percebe-se que há outros registros em ambas as tabelas, mas que tenham dados correspondentes em ambas, só há dois.

# MINUS

- Este operador permite obter os dados consultados com a 1º **SELECT** que não constem no resultado do 2º.



- É possível consultar dados de duas tabelas diferentes, mas a quantidade e tipo das colunas precisa ser compatível.

# MINUS - Exemplo

- Neste exemplo, vamos perceber que são selecionados os dados da tabela **Employee** que não constam na tabela **Job\_history**.

```
SELECT employee_id, job_id  
FROM employees  
MINUS  
SELECT employee_id, job_id  
FROM job_history  
ORDER BY employee_id;
```



|    | EMPLOYEE_ID | JOB_ID   |
|----|-------------|----------|
| 73 | 172         | SA_REP   |
| 74 | 173         | SA_REP   |
| 75 | 174         | SA_REP   |
| 76 | 175         | SA_REP   |
| 77 | 177         | SA_REP   |
| 78 | 178         | SA_REP   |
| 79 | 179         | SA_REP   |
| 80 | 180         | SH_CLERK |
| 81 | 181         | SH_CLERK |
| 82 | 182         | SH_CLERK |
| 83 | 183         | SH_CLERK |

# INSERT usando SELECT

- É possível inserir um conjunto de registros em uma tabela utilizando um SELECT. Exemplo:

```
CREATE TABLE funcionario (  
    id_funcionario NUMBER(6) PRIMARY KEY,  
    nm_funcionario VARCHAR2(45) NOT NULL,  
    ds_email VARCHAR2(30)  
);  
SELECT * FROM funcionario;
```

```
INSERT INTO funcionario (  
    SELECT employee_id, first_name||' '||last_name, email  
    FROM employees WHERE department_id=30  
);  
DELETE FROM funcionario;
```



# Criando grupos

- Anteriormente vimos o uso das funções de grupo: AVG, SUM, MAX, MIN e COUNT.
- Naquele momento as funções consideraram o conjunto de todos os registros da consulta para seu processamento, podemos dizer que este conjunto de registro é um grupo.
- Agora iremos ver como criar outro agrupamentos com base nos dados que estão sendo consultados.

# Cláusula GROUP BY

```
SELECT department_id,  
        AVG(salary)  
FROM employees  
GROUP BY department_id;
```

|   | DEPARTMENT_ID | AVG(SALARY)              |
|---|---------------|--------------------------|
| 1 | 100           | 8601,3333333333333333... |
| 2 | 30            | 4150                     |
| 3 | (null)        | 7000                     |
| 4 | 90            | 19333,333333333333333... |
| 5 | 20            | 9500                     |
| 6 | 70            | 10000                    |
| 7 | 110           | 10154                    |
| 8 | 50            | 3475,555555555555555...  |
| 9 | 80            | 8955,8823529411764705... |

- Para usarmos esta cláusula precisamos conhecer algumas regras:
  - Todos os campos do SELECT que não forem uma função de grupo devem aparecer no GROUP BY;
  - Os campos que aparecem no GROUP BY não precisam necessariamente aparecer no SELECT;

# Cláusula GROUP BY

```
SELECT AVG(salary)
FROM employees
GROUP BY department_id;
```

- Podemos usar o ORDER BY

|    | DEPARTMENT_ID | AVG(SALARY)    |
|----|---------------|----------------|
| 1  | 10            | 4400           |
| 2  | 20            | 9500           |
| 3  | 30            | 4150           |
| 4  | 40            | 6500           |
| 5  | 50            | 3475,555555... |
| 6  | 60            | 5760           |
| 7  | 70            | 10000          |
| 8  | 80            | 8955,882352... |
| 9  | 90            | 19333,33333... |
| 10 | 100           | 8601,333333... |

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

|   | AVG(SALARY)     |
|---|-----------------|
| 1 | 8601,333333...  |
| 2 | 4150            |
| 3 | 7000            |
| 4 | 19333,33333...  |
| 5 | 9500            |
| 6 | 10000           |
| 7 | 10154           |
| 8 | 3475,555555...  |
| 9 | 8955,8823529... |

# Cláusula GROUP BY

- Podemos agrupar mais de um campo;
- Observe que são somados os salários agrupados

```
SELECT department_id "Depto", job_id "Cargo", SUM(salary) "Soma"  
FROM employees  
GROUP BY department_id, job_id  
ORDER BY department_id;
```

|    | Depto | Cargo    | SALARY |
|----|-------|----------|--------|
| 1  | 10    | AD_ASST  | 4400   |
| 2  | 20    | MK_MAN   | 13000  |
| 3  | 20    | MK_REP   | 6000   |
| 4  | 30    | PU_MAN   | 11000  |
| 5  | 30    | PU_CLERK | 3100   |
| 6  | 30    | PU_CLERK | 2900   |
| 7  | 30    | PU_CLERK | 2800   |
| 8  | 30    | PU_CLERK | 2600   |
| 9  | 30    | PU_CLERK | 2500   |
| 10 | 40    | HR_REP   | 6500   |

|    | Depto | Cargo    | Soma  |
|----|-------|----------|-------|
| 1  | 10    | AD_ASST  | 4400  |
| 2  | 20    | MK_MAN   | 13000 |
| 3  | 20    | MK_REP   | 6000  |
| 4  | 30    | PU_CLERK | 13900 |
| 5  | 30    | PU_MAN   | 11000 |
| 6  | 40    | HR_REP   | 6500  |
| 7  | 50    | SH_CLERK | 64300 |
| 8  | 50    | ST_CLERK | 55700 |
| 9  | 50    | ST_MAN   | 36400 |
| 10 | 60    | IT_PROG  | 28800 |

# Cláusula GROUP BY

- Podemos restringir os departamentos que serão considerados no SELECT usando o WHERE. Fazendo assim, eles não serão considerados na definição dos

```
SELECT department_id, ROUND(AVG(Salary))  
FROM Employees  
WHERE department_id NOT IN (10,20,30)  
GROUP BY department_id  
ORDER BY department_id;
```

|   | DEPARTMENT_ID | ROUND(AVG(SAL...) |
|---|---------------|-------------------|
| 1 | 40            | 6500              |
| 2 | 50            | 3475,56           |
| 3 | 60            | 5760              |
| 4 | 70            | 10000             |
| 5 | 80            | 8955,88           |
| 6 | 90            | 19333,33          |
| 7 | 100           | 8601,33           |
| 8 | 110           | 10154             |

# Cláusula HAVING

- Ela é utilizada para definir restrições dentro do agrupamento definido pelo GROUP BY, ou seja, nela definimos a condição para que os valores sejam ou não considerados dentro do grupo.
- Cuidado para não confundir com a WHERE, nesta não podemos usar funções de grupo

```
SELECT department_id, AVG(salary)  
FROM employees  
WHERE AVG(salary) > 8000  
GROUP BY department_id;
```



```
SELECT department_id, AVG(salary)  
FROM employees  
GROUP BY department_id  
HAVING AVG(salary) > 8000;
```



# Cláusula HAVING

- A cláusula HAVING deve ser colocada logo após o GROUP BY;

```
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id
HAVING MAX(salary) > 10000;
```

|   | DEPARTMENT_ID | MAX(SALARY) |
|---|---------------|-------------|
| 1 | 100           | 12008       |
| 2 | 30            | 11000       |
| 3 | 90            | 24000       |
| 4 | 20            | 13000       |
| 5 | 110           | 12008       |
| 6 | 80            | 14000       |

|    | DEPARTMENT_ID | SALARY |
|----|---------------|--------|
| 2  | 110           | 12008  |
| 3  | 110           | 8300   |
| 4  | 100           | 12008  |
| 5  | 100           | 9000   |
| 6  | 100           | 8200   |
| 7  | 100           | 7800   |
| 8  | 100           | 7700   |
| 9  | 100           | 6900   |
| 10 | 90            | 24000  |
| 11 | 90            | 17000  |
| 12 | 90            | 17000  |
| 13 | 80            | 14000  |
| 14 | 80            | 13500  |
| 15 | 80            | 12000  |
| 16 | 80            | 11500  |
| 17 | 80            | 11000  |
| 18 | 80            | 11000  |
| 19 | 80            | 10500  |
| 20 | 80            | 10500  |

# Cláusula HAVING

- **Outro exemplo:** mostrar a média salarial de todos os departamentos que tiverem a média maior que 6000

```
SELECT department_id,  
        AVG(salary)  
FROM Employees  
GROUP BY department_id  
HAVING AVG(salary) > 6000  
ORDER BY department_id;
```

|   | DEPARTMENT_ID | AVG(SALARY)           |
|---|---------------|-----------------------|
| 1 | 20            | 9500                  |
| 2 | 40            | 6500                  |
| 3 | 70            | 10000                 |
| 4 | 80            | 8955,8823529411764... |
| 5 | 90            | 19333,333333333333... |
| 6 | 100           | 8601,333333333333...  |
| 7 | 110           | 10154                 |
| 8 | (null)        | 7000                  |



# Usando Cláusula WHERE e HAVING

- Neste exemplo é usado o WHERE para limitar a consulta aos departamentos 10, 20, 30 e 40 e o HAVING para determinar que somente serão incluídos os grupos que apresentarem os maiores salários acima de 10000.

```
SELECT department_id, MAX(salary)
FROM employees
WHERE department_id IN(10,20,30,40)
GROUP BY department_id
HAVING MAX(salary) > 10000
ORDER BY department_id;
```

```
SELECT department_id, MAX(salary)
FROM employees
WHERE department_id IN(10,20,30,40)
GROUP BY department_id
ORDER BY department_id;
```



|   | DEPARTMENT_ID | MAX(SALARY) |
|---|---------------|-------------|
| 1 | 20            | 13000       |
| 2 | 30            | 11000       |



|   | DEPARTMENT_ID | MAX(SALARY) |
|---|---------------|-------------|
| 1 | 10            | 4400        |
| 2 | 20            | 13000       |
| 3 | 30            | 11000       |
| 4 | 40            | 6500        |

# Usando Sub-consulta

- Este exemplo mostra os códigos de departamentos e os menores salários da tabela de empregados, **agrupados por departamento**, que tenha o **menor salário maior** que o menor salário dos funcionários do **departamento 50**.

```
SELECT department_id, MIN(salary)
FROM employees
GROUP BY department_id
HAVING MIN(salary) >
    (SELECT MIN(salary)
     FROM employees
     WHERE department_id = 50);
```

|    | DEPARTMENT_ID | MIN(SALARY) |
|----|---------------|-------------|
| 1  | 100           | 6900        |
| 2  | 30            | 2500        |
| 3  | (null)        | 7000        |
| 4  | 90            | 17000       |
| 5  | 20            | 6000        |
| 6  | 70            | 10000       |
| 7  | 110           | 8300        |
| 8  | 80            | 6100        |
| 9  | 40            | 6500        |
| 10 | 60            | 4200        |
| 11 | 10            | 4400        |

# Ordem das cláusulas no SELECT

**SELECT**      coluna(s)

**FROM**        tabela(s)

**WHERE**       condição linha

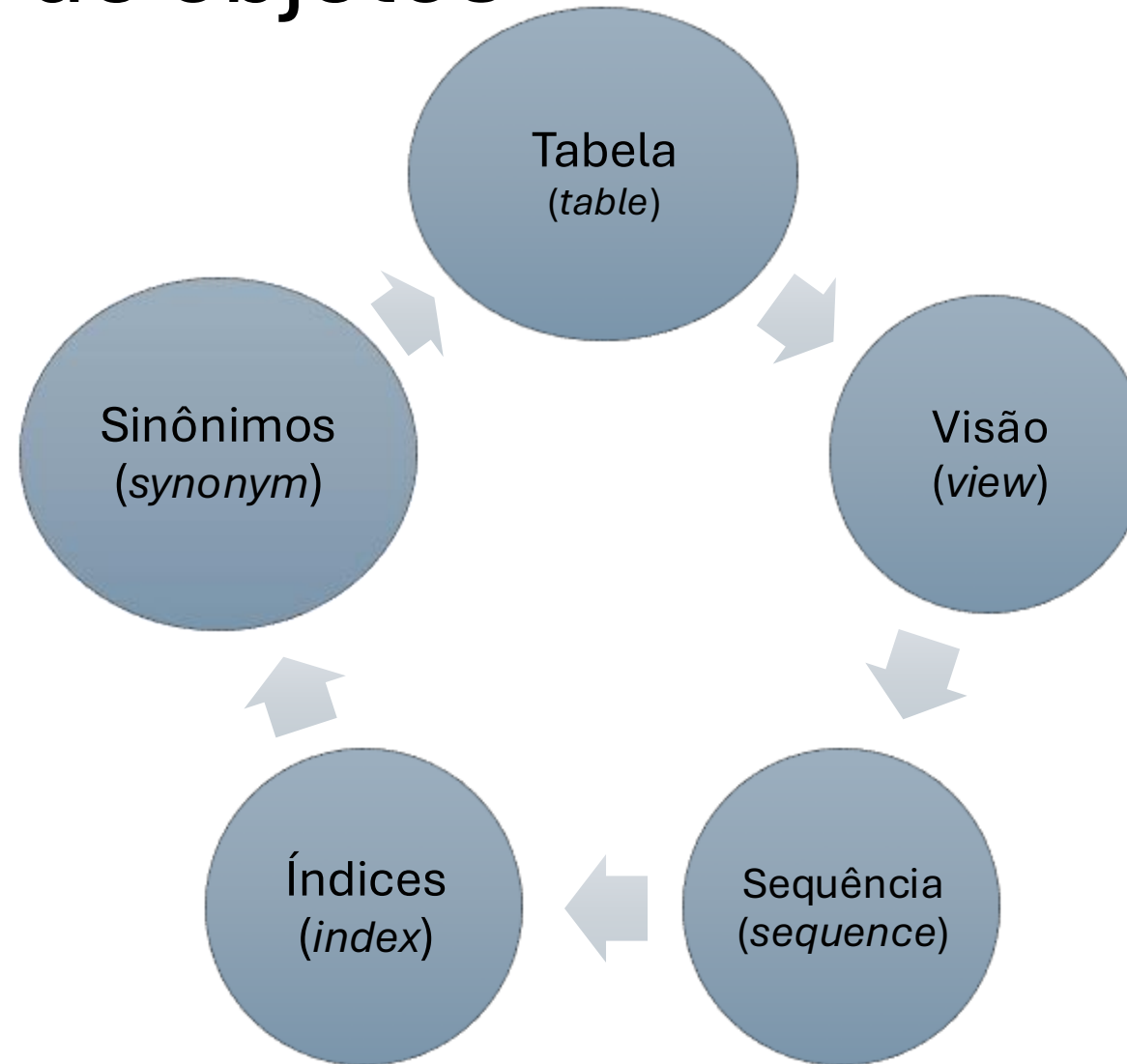
**GROUP BY**    coluna(s)

**HAVING**      condição de grupo de linhas

**ORDER BY**    coluna(s);

- **WHERE** - para estabelecer condições em linhas (SELECT)
- **GROUP BY** - para configurar/definir grupos
- **HAVING** - para estabelecer condições dentro dos grupos (GROUP BY)

# Tipos de objetos



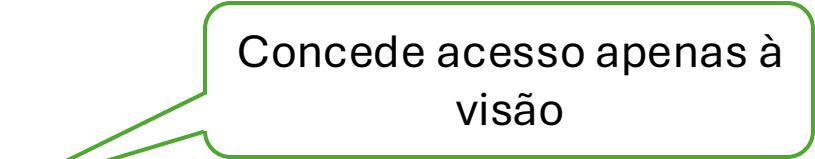
# VIEW

# Views (visões)

- É um tipo especial de visualização dos dados de uma ou várias tabelas;
- Ela é apenas uma visão de um grupo de colunas de uma ou mais tabelas de um grupo.
- São armazenadas no banco através de um comando select;

## Vantagens:

- Restringir o acesso aos dados => pode evitar que usuários não autorizados tenham acesso a todos os dados de uma tabela (segurança);
- Independência dos dados;
- Representar diferentes visões dos mesmos dados



Concede acesso apenas à  
visão

# Principais vantagens do uso de views



## EMP Table

| EMPNO | ENAME  | JOB       | MGR  | HIREDATE  | SAL  | COMM | DEPTNO |
|-------|--------|-----------|------|-----------|------|------|--------|
| 7839  | KING   | PRESIDENT |      |           |      |      | 20     |
| 7782  | CLARK  | MANAGER   | 7839 | 09-JAN-81 | 2400 |      | 20     |
| 7934  | MILLER | CLERK     | 7782 | 09-JAN-81 | 1300 |      | 20     |
| 7900  | JAMES  | CLERK     | 7698 | 03-DEC-81 | 950  |      | 30     |
| 7521  | WARD   | SALESMAN  | 7698 | 22-FEB-81 | 1250 | 500  | 30     |

## EMPVU10 View

| EMPNO | ENAME  | JOB       |
|-------|--------|-----------|
| 7839  | KING   | PRESIDENT |
| 7782  | CLARK  | MANAGER   |
| 7934  | MILLER | CLERK     |



# Views (visões)

## Sintaxe

```
CREATE VIEW nome_visão  
AS SELECT campos FROM tabela(s)  
WHERE condição
```

## Criação

```
CREATE VIEW visao01  
AS SELECT employee_id, last_name, salary  
      FROM employees  
      WHERE department_id = 80;
```

# Views (visões)

Com Alias

```
CREATE VIEW visao02  
AS SELECT employee_id ID_NUMBER,  
           last_name NAME, salary*12 ANN_SALARY  
FROM employees  
WHERE department_id = 50;
```

**DESCRIBE** visao01;

Sem suporte no Oracle live

**SELECT \* FROM** visao02;

Mostra o conteúdo de uma  
view

**SELECT \* FROM** visao01;

Sem suporte no Oracle live

# Modificando a View

Modificaremos a view **visao01** usando **CREATE OR REPLACE VIEW**. Adicionaremos apelidos, ALIAS para as colunas.

```
CREATE OR REPLACE VIEW visao01  
    (id_number, name, sal, department_id)  
AS SELECT employee_id, first_name || ' ' || last_name,  
           salary, department_id  
FROM employees  
    WHERE department_id = 80;
```



Alias

# Apagando uma View

Quando apagamos uma View, perdemos os dados?

```
DROP VIEW view;
```

```
DROP VIEW empvu80;
```

# Create View - complexas

Uma View com duas  
tabelas  
EMP e DEPT  
Ligação por **FK** (deptno  
da tabela EMP) e **PK**  
(deptno da tabela DEPT)

## Visão composta em duas tabelas (join view)

Tabelas: DEPT e EMP

Criar uma visão usando colunas de ambas as tabelas;  
Como existem duas é necessário especificar o nome de  
cada tabela antes do nome da coluna.

```
create view empdep_visao  
as select emp.ename, emp.deptno, dept.dname  
from scott.dept, scott.emp  
where dept.deptno = emp.deptno;
```

```
select * from empdep_visao;
```

| ENAME  | DEPTNO | DNAME      |
|--------|--------|------------|
| CLARK  | 10     | ACCOUNTING |
| MILLER | 10     | ACCOUNTING |

## Create View – complexas (2)

```
Create View dep_soma
(nome, menor_sal, maior_sal, media_sal)
AS Select d.dname, min(e.sal), max (e.sal),
avg(e.sal)
From scott.emp e, scott.dept d
where e.deptno = d.deptno
Group by d.dname;
```

```
Select * from dep_soma;
```

| NOME       | MENOR_SAL | MAIOR_SAL | MEDIA_SAL                         |
|------------|-----------|-----------|-----------------------------------|
| RESEARCH   | 800       | 3000      | 2175                              |
| SALES      | 950       | 2850      | 1566.6666666666666666666666666667 |
| ACCOUNTING | 1300      | 5000      | 2916.6666666666666666666666666667 |

[Download CSV](#)

3 rows selected.

Uma View com duas tabelas  
EMP e DEPT  
Além da utilização do  
GROUP BY  
Vejam que o agrupamento é  
pelo nome do  
departamento, pois estamos  
utilizando duas tabelas. Fica  
mais claro para os usuários.  
(com uma tabela, o  
agrupamento era pelo código)

# Criando View Complexa

- Podemos criar views com *N tabelas*, e utilizando inclusive *funções de grupo*.

```
CREATE OR REPLACE VIEW dept_sum_vu  
  (name, minsal, maxsal, avgsal)  
AS SELECT  d.department_name, MIN(e.salary),  
            MAX(e.salary),AVG(e.salary)  
FROM    employees e JOIN departments d  
ON      (e.department_id = d.department_id)  
GROUP BY d.department_name;
```

JOIN



# **Ligações**

## **Extraindo Dados de Mais de uma Tabela**

### **Ligações (Joins)**

Uma ligação é usada quando a pesquisa SQL requer dados de mais de uma tabela do Banco de Dados.

Linhas em uma tabela devem ser ligadas a outras linhas de acordo com o valor comum existente na coluna correspondente.

Existem três tipos principais de condições de ligações:

- 1) Equi-join
- 2) Non-equi-join
- 3) Outer-Join

# Ligações

## Extraindo Dados de Mais de uma Tabela

Para as próximas atividades, iremos utilizar mais de uma tabela para extrair dados.

No próximo slide, está o DER das tabelas do usuário HR no Oracle e seus relacionamentos.

# Natural Join

- É baseada em todos os campos de mesmo nome em 2 tabelas.
  - Se os nomes dos campos e os tipos de dados forem iguais então retorna a consulta com sucesso;



- Se os nomes forem iguais porém os tipos de dados forem diferentes, retorna erro.



## Recuperando registros: Natural Join

```
SELECT department_id, department_name,  
       location_id, city  
FROM   departments  
NATURAL JOIN locations ;
```

| DEPARTMENT_ID | DEPARTMENT_NAME | LOCATION_ID | CITY                |
|---------------|-----------------|-------------|---------------------|
| 60            | IT              | 1400        | Southlake           |
| 50            | Shipping        | 1500        | South San Francisco |
| 10            | Administration  | 1700        | Seattle             |
| 90            | Executive       | 1700        | Seattle             |
| 110           | Accounting      | 1700        | Seattle             |
| 190           | Contracting     | 1700        | Seattle             |
| 20            | Marketing       | 1800        | Toronto             |
| 80            | Sales           | 2500        | Oxford              |

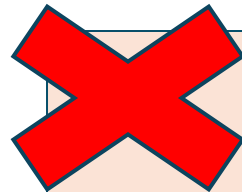
8 rows selected.

```
SELECT  department_id, department_name,  
        location_id, city  
FROM    departments  
NATURAL JOIN locations  
WHERE   department_id IN (20, 50);
```

## Natural Join + USING

- Quando temos várias colunas com o mesmo nome e tipos diferentes, podemos escolher o campo de junção através do (USING).
- Não use alias de tabela ou campo no using e no where, pois lembre-se: o campo deve ser comum às duas tabelas.

```
SELECT l.city, d.department_name  
FROM   locations l JOIN departments d  
USING (location_id)  
WHERE  location_id = 1400;
```



```
SELECT l.city, d.department_name  
FROM locations l JOIN departments d USING  
(location_id)  
WHERE d.location_id = 1400;
```

# EquiJoin

## EquiJoin

Para descobrir, qual departamento os empregados estão, nós comparamos a coluna Department\_ID da tabela Employees com o mesmo valor de Department\_ID na tabela Departments. O relacionamento entre a tabela Employees e a Departments é um equi-join, em que o valor da coluna Department\_ID seja igual para ambas as tabelas.

Uma condição de ligação é especificada na cláusula WHERE:

```
SELECT      coluna(s)
FROM        tabela(s)
WHERE       condição de ligação
```

## Extraindo Dados de Mais de uma Tabela – Equi-Join

Para ligar as duas tabelas **Employees** e **Departments**, faça:

```
SELECT Employees.Employee_ID, Employees.Department_ID, Department_Name
```

```
FROM Employees, Departments
```

```
WHERE Employees.Department_ID = Departments.Department_ID;
```

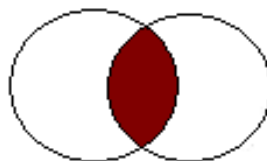
Ligação das  
tabelas (JOIN)

**EMPLOYEES**

| EMPLOYEE_ID | LAST_NAME | DEPARTMENT_ID |
|-------------|-----------|---------------|
| 100         | King      | 90            |
| 101         | Kochhar   | 90            |
| ...         |           |               |
| 202         | Fay       | 20            |
| 205         | Higgins   | 110           |
| 206         | Gietz     | 110           |

**DEPARTMENTS**

| DEPARTMENT_ID | DEPARTMENT_NAME | LOCATION_ID |
|---------------|-----------------|-------------|
| 10            | Administration  | 1700        |
| 20            | Marketing       | 1800        |
| 50            | Shipping        | 1900        |
| 60            | IT              | 1400        |
| 80            | Sales           | 2500        |
| 90            | Executive       | 1700        |
| 110           | Accounting      | 1700        |
| 190           | Contracting     | 1700        |



**Inner Join**

| EMPLOYEE_ID | DEPARTMENT_ID | DEPARTMENT_NAME |
|-------------|---------------|-----------------|
| 200         | 10            | Administration  |
| 201         | 20            | Marketing       |
| 202         | 20            | Marketing       |
| ...         |               |                 |
| 102         | 90            | Executive       |
| 205         | 110           | Accounting      |
| 206         | 110           | Accounting      |

## Extraindo Dados de Mais de uma Tabela – Equi-Join

Para ligar as duas tabelas Employees e Departments utilizando JOIN, faça:

```
SELECT Employees.Employee_ID,  
           Employees.Department_ID, Department_Name  
FROM   Employees JOIN Departments  
ON     Employees.Department_ID = Departments.Department_ID;
```

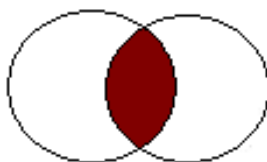
Ligação das  
tabelas (JOIN)

EMPLOYEES

| EMPLOYEE_ID | LAST_NAME | DEPARTMENT_ID |
|-------------|-----------|---------------|
| 100         | King      | 90            |
| 101         | Kochhar   | 90            |
| ...         |           |               |
| 202         | Fay       | 20            |
| 205         | Higgins   | 110           |
| 206         | Gietz     | 110           |

DEPARTMENTS

| DEPARTMENT_ID | DEPARTMENT_NAME | LOCATION_ID |
|---------------|-----------------|-------------|
| 10            | Administration  | 1700        |
| 20            | Marketing       | 1800        |
| 50            | Shipping        | 1900        |
| 60            | IT              | 1400        |
| 80            | Sales           | 2500        |
| 90            | Executive       | 1700        |
| 110           | Accounting      | 1700        |
| 190           | Contracting     | 1700        |



Inner Join

| EMPLOYEE_ID | DEPARTMENT_ID | DEPARTMENT_NAME |
|-------------|---------------|-----------------|
| 200         | 10            | Administration  |
| 201         | 20            | Marketing       |
| 202         | 20            | Marketing       |
| ...         |               |                 |
| 102         | 90            | Executive       |
| 205         | 110           | Accounting      |
| 206         | 110           | Accounting      |



## Utilizando Alias – “Apelidos”

- O ‘Alias’ também pode ser utilizado no comando select para apelidar tabelas conforme exemplo abaixo.
- O uso do alias simplifica a consulta;

```
SELECT e.employee_id, e.last_name,  
       d.location_id, department_id  
FROM   employees e JOIN departments d  
USING (department_id) ;
```

## Comparação

```
SELECT Employees.Employee_ID,  
       Employees.Department_ID, Department_Name  
FROM   Employees, Departments  
WHERE  Employees.Department_ID = Departments.Department_ID
```

```
SELECT Employees.Employee_ID,  
       Employees.Department_ID, Department_Name  
FROM   Employees JOIN Departments  
       ON Employees.Department_ID = Departments.Department_ID
```

```
SELECT E.Employee_ID,  
       Department_ID, D.Department_Name  
FROM   Employees E JOIN Departments D  
       USING (department_id)
```

# Ligações

## Extraindo Dados de Mais de uma Tabela - Equi-Join

Utilizando Join e condições juntas:

```
SELECT E.First_Name, E.Last_Name, D.Department_Name  
FROM   Employees E, Departments D  
WHERE  E.Department_ID =D.Department_ID  
        AND E.Department_ID = 30;
```

| FIRST_NAME | LAST_NAME  | DEPARTMENT_NAME |
|------------|------------|-----------------|
| Den        | Raphaely   | Purchasing      |
| Alexander  | Khoo       | Purchasing      |
| Shelli     | Baida      | Purchasing      |
| Sigal      | Tobias     | Purchasing      |
| Guy        | Himuro     | Purchasing      |
| Karen      | Colmenares | Purchasing      |

Utilizando Join e condições separadas:

```
SELECT E.First_Name, E.Last_Name, D.Department_Name  
FROM   Employees E JOIN Departments D  
ON      E.Department_ID =D.Department_ID  
WHERE  E.Department_ID = 30;
```

# Ligações

## Extraindo Dados de Mais de uma Tabela - Equi-Join

Caso não seja realizado o relacionamento, o SQL **não está errado mas o resultado estará errado.**

**Portanto:** na ausência da condição WHERE (relacionamento), cada linha da tabela Employees estará ligada com cada linha de Departments ...

Caso a tabela Employees tenha 14 registros e a tabela Departments tenha 4 linhas, será feito o **produto cartesiano (14X4=56)**, e retornará 56 registros.

**A EXIBIÇÃO DESSES DADOS ESTARÁ ERRADO!!!**

# Self Join

**EMPLOYEES (WORKER)**

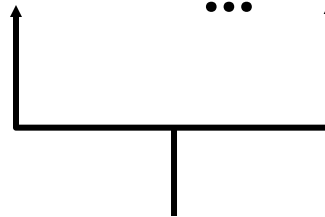
| EMPLOYEE_ID | LAST_NAME | MANAGER_ID |
|-------------|-----------|------------|
| 100         | King      |            |
| 101         | Kochhar   | 100        |
| 102         | De Haan   | 100        |
| 103         | Hunold    | 102        |
| 104         | Ernst     | 103        |
| 107         | Lorentz   | 103        |
| 124         | Mourgos   | 100        |

...

**EMPLOYEES (MANAGER)**

| EMPLOYEE_ID | LAST_NAME |
|-------------|-----------|
| 100         | King      |
| 101         | Kochhar   |
| 102         | De Haan   |
| 103         | Hunold    |
| 104         | Ernst     |
| 107         | Lorentz   |
| 124         | Mourgos   |

...



**MANAGER\_ID da tabela WORKER é igual ao EMPLOYEE\_ID da tabela MANAGER.**

# Self Join

```
SELECT e.last_name emp, m.last_name mgr
FROM   employees e JOIN employees m
ON     (e.manager_id = m.employee_id);
```

Ou

```
SELECT e.last_name emp, m.last_name mgr
FROM   employees e, employees m
WHERE  e.manager_id = m.employee_id;
```

| EMP       | MGR  |
|-----------|------|
| Hartstein | King |
| Zlotkey   | King |
| Mourgos   | King |
| De Haan   | King |
| Kochhar   | King |
| ...       |      |

19 rows selected.

# Join entre très Tabelas

```
SELECT employee_id, city, department_name
FROM   employees e
JOIN   departments d
ON     d.department_id = e.department_id
JOIN   locations l
ON     d.location_id = l.location_id;
```

Ou

```
SELECT employee_id, city, department_name
FROM   employees e, departments d, locations l
WHERE  d.department_id = e.department_id AND
       d.location_id = l.location_id;
```

| EMPLOYEE_ID | CITY                | DEPARTMENT_NAME |
|-------------|---------------------|-----------------|
| 103         | Southlake           | IT              |
| 104         | Southlake           | IT              |
| 107         | Southlake           | IT              |
| 124         | South San Francisco | Shipping        |
| 141         | South San Francisco | Shipping        |
| 142         | South San Francisco | Shipping        |
| 143         | South San Francisco | Shipping        |
| 144         | South San Francisco | Shipping        |

••••  
19 rows selected.

# NonEquijoins

**EMPLOYEES**

| LAST_NAME | SALARY |
|-----------|--------|
| King      | 24000  |
| Kochhar   | 17000  |
| De Haan   | 17000  |
| Hunold    | 9000   |
| Ernst     | 6000   |
| Lorentz   | 4200   |
| Mourgos   | 5800   |
| Rajs      | 3500   |
| Davies    | 3100   |
| Matos     | 2600   |
| Vargas    | 2500   |
| Zlotkey   | 10500  |
| Abel      | 11000  |
| Taylor    | 8600   |

...

20 rows selected.

**JOB\_GRADES**

| GRA | LOWEST_SAL | HIGHEST_SAL |
|-----|------------|-------------|
| A   | 1000       | 2999        |
| B   | 3000       | 5999        |
| C   | 6000       | 9999        |
| D   | 10000      | 14999       |
| E   | 15000      | 24999       |
| F   | 25000      | 40000       |

← O salário dos empregados deve estar compreendido entre o menor e o maior salário da tabela JOB\_GRADES.



## Recuperando Registros com NonEquijoins

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e JOIN job_grades j
ON     e.salary
      BETWEEN j.lowest_sal AND j.highest_sal;
```

Ou

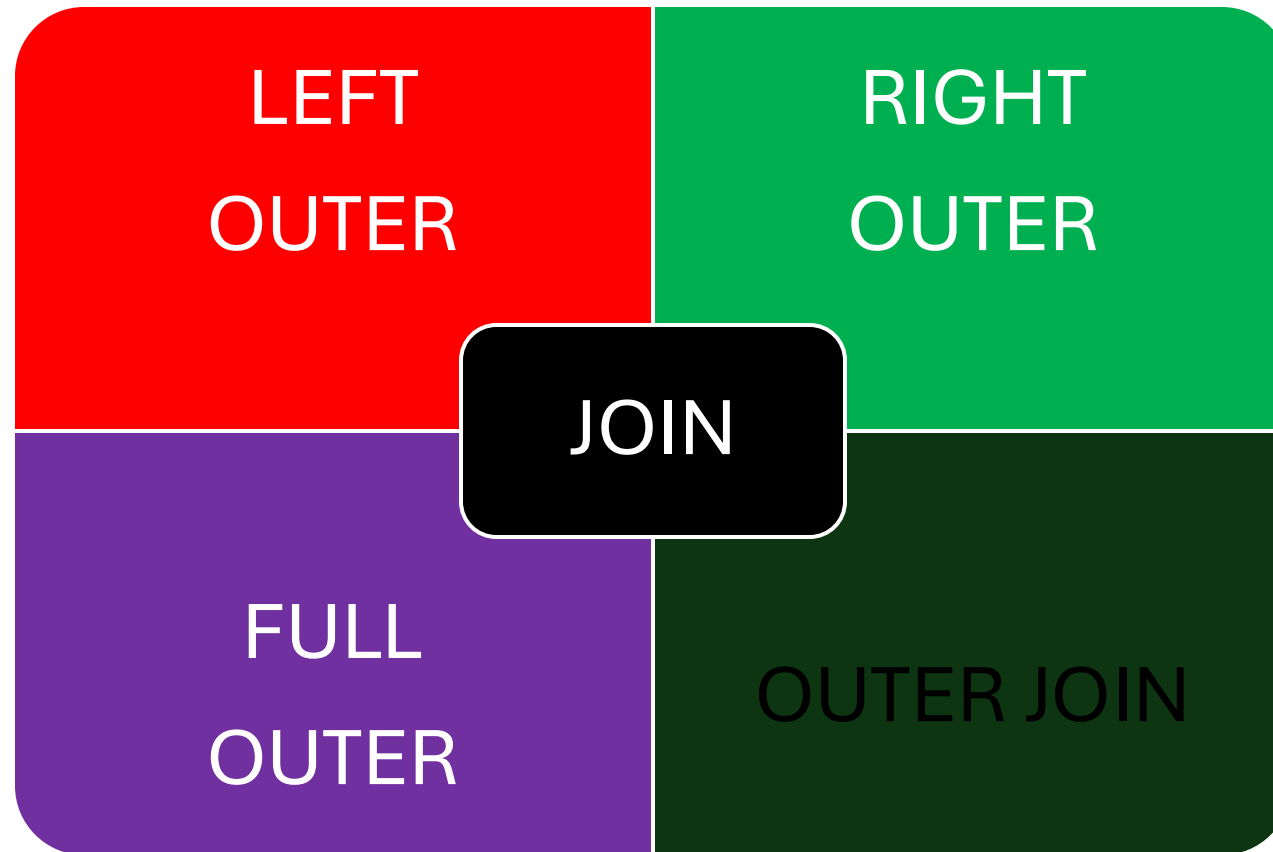
```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e, job_grades j
WHERE  e.salary BETWEEN j.lowest_sal AND
      j.highest_sal;
```

| LAST_NAME | SALARY | GRA |
|-----------|--------|-----|
| Matos     | 2600   | A   |
| Vargas    | 2500   | A   |
| Lorentz   | 4200   | B   |
| Mourgos   | 5800   | B   |
| Rajs      | 3500   | B   |
| Davies    | 3100   | B   |
| Whalen    | 4400   | B   |
| Hunold    | 9000   | C   |
| Ernst     | 6000   | C   |

...

20 rows selected.

## LEFT OUTER, RIGHT OUTER e FULL OUTER JOIN



# Outer Join

Além de mostrar registros cujos campos em comum estejam presentes nas duas tabelas, **ainda mostram os que faltam.**

- Neste caso, a tabela DEPARTMENTS possui o departamento de Contracting que não possui nenhum EMPLOYEES alocado.
- No caso de uma consulta na tabela de EMPLOYEES o departamento 190 é o único que não aparecerá.

**DEPARTMENTS**

| DEPARTMENT_NAME | DEPARTMENT_ID |
|-----------------|---------------|
| Administration  | 10            |
| Marketing       | 20            |
| Shipping        | 50            |
| IT              | 60            |
| Sales           | 80            |
| Executive       | 90            |
| Accounting      | 110           |
| Contracting     | 190           |

8 rows selected.

**EMPLOYEES**

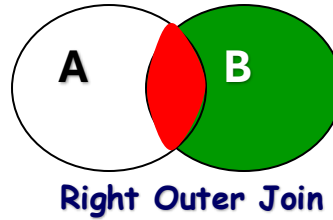
| DEPARTMENT_ID | LAST_NAME |
|---------------|-----------|
| 90            | King      |
| 90            | Kochhar   |
| 90            | De Haan   |
| 60            | Hunold    |
| 60            | Ernst     |
| 60            | Lorentz   |
| 50            | Mourgos   |
| 50            | Rajs      |
| 50            | Davies    |
| 50            | Matos     |
| 50            | Vargas    |
| 80            | Zlotkey   |

...

20 rows selected.

**Não há empregados no departamento 190.**

# Right Outer Join



- Apresenta os departamentos que tem e que não tem funcionários.

```
SELECT e.last_name, d.department_id, d.department_name  
FROM   employees e RIGHT OUTER JOIN departments d  
ON     (e.department_id = d.department_id) ;
```

...

| LAST_NAME | DEPARTMENT_ID | DEPARTMENT_NAME |
|-----------|---------------|-----------------|
| Whalen    | 10            | Administration  |
| Fay       | 20            | Marketing       |
| Hartstein | 20            | Marketing       |
| Davies    | 50            | Shipping        |
| ...       |               |                 |
| Kochhar   | 90            | Executive       |
| Gietz     | 110           | Accounting      |
| Higgins   | 110           | Accounting      |
|           | 190           | Contracting     |

20 rows selected.

# Right Outer Join (+)

## Usando o Operador (+)

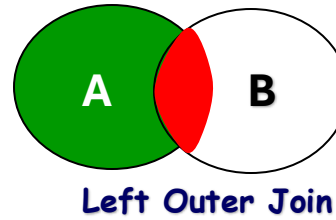
O símbolo (+) é o operador para junção externa , ou seja, se um dado existir somente em uma tabela, com este símbolo será possível retornar linha(s) faltando o outro valor.

O símbolo (+) deve ser inserido junto ao nome da tabela que não tem linhas correspondentes.

```
SELECT E.First_Name, E.Last_Name,  
       D.Department_Name  
FROM   Employees E, Departments D  
WHERE  E.Department_ID (+) =  
       D.Department_ID  
ORDER BY D.Department_Name;
```

| FIRST_NAME | LAST_NAME | DEPARTMENT_NAME    |
|------------|-----------|--------------------|
| William    | Gietz     | Accounting         |
| Shelley    | Higgins   | Accounting         |
| Jennifer   | Whalen    | Administration     |
| -          | -         | Benefits           |
| -          | -         | Construction       |
| -          | -         | Contracting        |
| -          | -         | Control And Credit |
| -          | -         | Corporate Tax      |
| Lex        | De Haan   | Executive          |
| Neena      | Kochhar   | Executive          |

# Left Outer Join



- Apresenta os empregados que tem e que não tem departamentos.

```
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e LEFT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```

| LAST_NAME | DEPARTMENT_ID | DEPARTMENT_NAME |
|-----------|---------------|-----------------|
| Whalen    | 10            | Administration  |
| Fay       | 20            | Marketing       |
| Hartstein | 20            | Marketing       |
| ...       |               |                 |
| De Haan   | 90            | Executive       |
| Kochhar   | 90            | Executive       |
| King      | 90            | Executive       |
| Gietz     | 110           | Accounting      |
| Higgins   | 110           | Accounting      |
| Grant     |               |                 |

20 rows selected.

# Left Outer Join

## Usando o Operador (+)

```
SELECT E.First_Name, E.Last_Name, D.Department_Name  
FROM   Employees E, Departments D  
WHERE  E.Department_ID = D.Department_ID (+)  
ORDER BY D.Department_Name
```

| FIRST_NAME | LAST_NAME | DEPARTMENT_NAME |
|------------|-----------|-----------------|
| Shelley    | Higgins   | Accounting      |
| William    | Gietz     | Accounting      |
| Jennifer   | Whalen    | Administration  |
| Steven     | King      | Executive       |
| Neena      | Kochhar   | Executive       |
| Lex        | De Haan   | Executive       |
| Daniel     | Faviet    | Finance         |

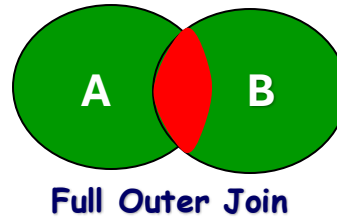
● ● ●

|           |            |          |
|-----------|------------|----------|
| Ki        | Gee        | Shipping |
| Hazel     | Philtanker | Shipping |
| Renske    | Ladwig     | Shipping |
| Stephen   | Stiles     | Shipping |
| John      | Seo        | Shipping |
| Joshua    | Patel      | Shipping |
| Trenna    | Rajs       | Shipping |
| Curtis    | Davies     | Shipping |
| Randall   | Matos      | Shipping |
| Jean      | Fleaur     | Shipping |
| Kimberely | Grant      | -        |

106 linhas retornadas em 0,00 segundos

[Exportação](#)

# Full Outer Join



- Retorna um Right e um Left Join ao mesmo tempo.

```
SELECT e.last_name, d.department_id, d.department_name
FROM   employees e FULL OUTER JOIN departments d
ON     (e.department_id = d.department_id);
```

| LAST_NAME | DEPARTMENT_ID | DEPARTMENT_NAME |
|-----------|---------------|-----------------|
| Whalen    | 10            | Administration  |
| Fay       | 20            | Marketing       |
| Hartstein | 20            | Marketing       |
| ...       |               |                 |
| King      | 90            | Executive       |
| Gietz     | 110           | Accounting      |
| Higgins   | 110           | Accounting      |
| Grant     |               |                 |
|           | 190           | Contracting     |

21 rows selected.



# Produto Cartesiano

**EMPLOYEES (20 rows)**

| EMPLOYEE_ID | LAST_NAME | DEPARTMENT_ID |
|-------------|-----------|---------------|
| 100         | King      | 90            |
| 101         | Kochhar   | 90            |
| ...         |           |               |
| 202         | Fay       | 20            |
| 205         | Higgins   | 110           |
| 206         | Gietz     | 110           |

20 rows selected.

**DEPARTMENTS (8 rows)**

| DEPARTMENT_ID | DEPARTMENT_NAME | LOCATION_ID |
|---------------|-----------------|-------------|
| 10            | Administration  | 1700        |
| 20            | Marketing       | 1800        |
| 50            | Shipping        | 1500        |
| 60            | IT              | 1400        |
| 80            | Sales           | 2500        |
| 90            | Executive       | 1700        |
| 110           | Accounting      | 1700        |
| 190           | Contracting     | 1700        |

8 rows selected.

**Produto  
cartesiano:  
20 x 8 = 160 rows**

| EMPLOYEE_ID | DEPARTMENT_ID | LOCATION_ID |
|-------------|---------------|-------------|
| 100         | 90            | 1700        |
| 101         | 90            | 1700        |
| 102         | 90            | 1700        |
| 103         | 60            | 1700        |
| 104         | 60            | 1700        |
| 107         | 60            | 1700        |

...  
160 rows selected.

# Cross Join

O Cross Join retorna o produto cartesiano de 2 tabelas.

```
SELECT last_name, department_name  
FROM employees  
CROSS JOIN departments;
```

| LAST_NAME | DEPARTMENT_NAME |
|-----------|-----------------|
| King      | Administration  |
| Kochhar   | Administration  |
| De Haan   | Administration  |
| Hunold    | Administration  |

...

160 rows selected.

# **Agradecimentos**

**Slides baseados nas aulas da prof. Cristiane - Unicsul**